Diagnosing Visual Robustness in JEPA World Models through Action-Conditioned Predictive Consistency

June 16, 2026

Abstract

Latent predictive world models such as JEPAs predict future representations rather than pixels, but this alone does not define visual robustness for control. We argue that robustness should be measured as *action-conditioned predictive consistency*: corrupted and unperturbed views of the same state may encode differently, but under the same action sequence they should induce consistent task-relevant future predictions while preserving action-relevant distinctions. We study LeWorldModel (LeWM) on PushT, TwoRoom, Reacher, and Cube using 36 checkpoints and a unified 3-seed $\times$ 100-trajectory evaluation protocol; the corruption setting is a controlled diagnostic probe, not a proposed benchmark. Under observation-only Gaussian noise with an unperturbed goal, a no-noise LeWM checkpoint drops from 86.33% to 4.33% on PushT and from 58.67% to 18.33% on Reacher. Full-sequence input-side noise augmentation recovers much of this performance, but behaves as a coarse global pressure with task-dependent plateaus. A paired Gaussian-noise ACPC-basin diagnostic shows tighter same-action prediction radii on robust checkpoints (PushT R_F: 1.543 $\rightarrow$ 0.088). The contribution is diagnostic: a reframing, paired probes, and negative ablations that delimit simple fixes, not a new training algorithm.

Keywords: visual world models; JEPA; action-conditioned predictive consistency; visual robustness; predictive-dynamics diagnostics.

1 Introduction

1.1 Latent prediction is not a robustness definition

JEPAs predict future representations instead of reconstructing pixels, which can reduce pressure to encode high-magnitude irrelevant visual detail [2, 4, 35]. For control, however, "irrelevant" is action- and transition-dependent. A representation may ignore a background texture but must preserve a contact pose or doorway state if that changes the next useful action. Thus encoder-level invariance is an incomplete target: planning uses the model's *predictions*, so robustness must be stated after action-conditioned prediction.

Our controlled diagnostic probe makes this failure concrete; it is not proposed as a new robustness benchmark. A no-noise LeWM [23] checkpoint scores 86.33% on unperturbed PushT but only 4.33% under observation-only Gaussian noise at std 0.08 with the goal unperturbed; Reacher drops from 58.67% to 18.33%, and TwoRoom from 94.00% to 65.67%. Pixels+goal corruption is reported as a harder auxiliary stress condition. Visual-corruption fragility is not new [40, 39, 15, 24]; our focus is what it reveals about JEPA latent world-model control.

1.2 From visual invariance to action-conditioned predictive consistency

We reframe visual robustness for world-model control at the level of *action-conditioned predictive dynamics*. Let $z = E(h)$ and $\tilde{z} = E(\tilde{h})$ be the latents an encoder E produces from an unperturbed history h and a corrupted history $\tilde{h}$ of the same underlying state, and let $F^k(\cdot, \mathbf{a}_{0:k-1})$ be the action-conditioned latent predictor rolled out for k steps under an action sequence $\mathbf{a}$. We *allow* $z \neq \tilde{z}$. The robustness target is placed *after* the predictor: in task-relevant coordinates,

$$F^k(z, \mathbf{a}_{0:k-1}) \approx F^k(\tilde{z}, \mathbf{a}_{0:k-1}), \qquad k = 1, \dots, H, \tag{1}$$

so that the two branches induce the same next-state and rollout predictions under the same action intervention. The requirement is *selective*: it should hold only for nuisance perturbations of the same underlying state. State differences that change action-conditioned transitions, costs, or the optimal action must remain *discriminable*, or the planner loses the distinctions it needs. We make both halves precise in Section 3.

This reframing changes what counts as a robustness measurement. Encoder-level latent closeness, $\|z - \tilde{z}\| \to 0$, is neither necessary (a large but task-irrelevant encoder shift can wash out after the predictor) nor sufficient (a small encoder shift can still flip the planned action). Planning, cost, and action metrics remain useful, but as *downstream readouts* of predictive consistency rather than as the central definition; future objectives should regularise the action-conditioned predictive dynamics.

1.3 Existing corruption results as a controlled probe

A natural remedy is input-side noise augmentation. We sweep `std_max` $\in \{0.01, \dots, 0.08\}$ across four tasks and find recovery, but not a single optimal setting: TwoRoom benefits from heavy noise, PushT's unperturbed and noisy-eval optima dissociate, Reacher has a broad plateau, and Cube responds weakly. This pattern is consistent with a coarse global pressure, not with a selective objective that knows which visual variation is nuisance and which controls future dynamics. We therefore use the sweep and diagnostics to motivate predictive-consistency objectives. Appendix H records a negative target-view ablation: perturbed-history $\to$ original-future denoising is not a sufficient closed-loop fix, so the retained empirical mainline is the full-sequence perturbed-target branch.

1.4 Contributions

C1 — Problem reframing. We formulate visual robustness for world-model control as action-conditioned predictive consistency plus an action-relevant discriminability countercondition, rather than encoder-level invariance.

C2 — Diagnostic evidence. Across 36 LeWM checkpoints and a PLDM replication, we show severe corruption fragility, task-dependent recovery under input-side noise augmentation, and the limits of pointwise diagnostics: after conditioning on `std_max`, the strongest single-step fragility ratio does not explain the PushT corruption gap (partial Spearman $\rho = +0.19$, CI including 0).

C3 — Paired consistency probes and scope boundaries. We report dense Gaussian-noise ACPC basin diagnostics on all 36 LeWM checkpoints and all 36 PLDM checkpoints, while keeping the broader Phase-0 ACPC/PCC/CRA/MAF/ADM/SPRR sweep in Appendix G as exploratory evidence, not as a universal robustness predictor. We also use two negative ablations to constrain method claims: error-based heteroscedastic reweighting can erase action-critical transitions, and perturbed-history → original-future denoising is not a sufficient closed-loop fix. These are design constraints, not completed methods.

2 Related Work

2.1 JEPA and latent world models

JEPA [21] predicts in latent space rather than reconstructing pixels; I-JEPA [2] and V-JEPA [4, 3] extend this idea to images and video. LeWM [23], our baseline, stabilises an end-to-end JEPA world model with SIGReg [7]. Its Violation-of-Expectation result probes surprise under physical and colour changes, whereas we measure control success under pixel corruptions. PLDM [29, 30], via `stable-worldmodel` [22], supplies the second method family in Appendix E; the main diagnostic analysis remains LeWM-centred.

2.2 Joint-embedding, reconstruction, and augmentation alignment

Augmentation and joint-embedding losses impose invariances only when their training signal identifies the right nuisance factors [37, 32, 42]. Van Assel et al. [35] show that joint-embedding prediction can reduce pressure to encode high-magnitude irrelevant features, but still needs augmentation or bias to decide *which* variations are irrelevant. In control, that decision is action- and transition-conditioned. Seq-JEPA [11] addresses a related invariance–equivariance tension architecturally; we instead state the requirement on predictions.

2.3 LeJEPA identifiability and latent world-model theory

Klindt et al. [18] analyse when LeJEPA recovers latent world variables and supports latent planning. Our question is complementary: given a learned visual world model, when do unperturbed and corrupted views of the same state induce consistent action-conditioned predictions? We do not claim an identifiability result.

2.4 Structured action consistency

Wang et al. [38] formalise action-conditioned video world modeling as a group action and introduce identity, inverse, and composition-consistency metrics for action-faithful rollouts. That line is complementary to ours: group-action consistency tests whether different action sequences obey algebraic structure in generated dynamics, whereas ACPC tests whether paired corrupted and unperturbed views of the same state induce consistent predictive dynamics under a fixed action sequence, with a discriminability guard for action-distinct states.

2.5 Bisimulation and control-relevant representations

Bisimulation merges states with equivalent reward and transition behaviour. DeepMDP [10], DBC [41], bisimulation-regularized MPC [28], and Bisim-JEPA [34] use this principle to learn control-relevant representations. Our framing differs in that paired unperturbed/corrupted views of the *same* state need not share an encoder latent; we ask for agreement after the action-conditioned predictor while explicitly preserving action-distinct transitions. In this sense ACPC is a paired, corruption-conditioned local consistency diagnostic, not a learned bisimulation metric over arbitrary state pairs.

2.6 Visual robustness in model-based and pixel-based RL

DrQ/DrQ-v2 [40, 39] and SODA/DMC-GB [15] establish augmentation-based visual robustness for pixel RL. DreamerV3 [13], TD-MPC2 [14], and `stable-worldmodel` [22] study visual world-model control, but do not settle the JEPA predictive-consistency question. ViGMO [24] is closest: it studies unseen visual distractions and adds latent consistency. Our boundary is that ViGMO regularises generic encoder consistency, whereas we place consistency on action-conditioned rollouts and pair it with a discriminability guard. JEPA robustness work such as N-JEPA [25], the variational VJEPA of Huang [16] (distinct from V-JEPA [4]), and US-JEPA [26] is compatible with our result because noisy-environment representation recovery and closed-loop action optimisation test different properties.

2.7 Value-aware and value-equivalent model learning

Value-equivalent and value-aware models [12, 36] argue that a model should serve downstream control rather than fit all dynamics equally. VIBR [6] similarly favours view-invariant value functions over invariant representations. We adopt the downstream-consequence principle, but apply it to predictive dynamics: same-state views should agree after action-conditioned prediction, while different action-relevant transitions stay distinct.

2.8 Representation diagnostics and collapse analysis

We use standard representation diagnostics: effective rank [27, 17, 33], CKA [19], linear probes [1], and anti-collapse context from VICReg [5]. RankMe [9] motivates our cross-checkpoint question: after removing the sweep-level training-noise trend, does a label-free predictor-sensitivity diagnostic retain downstream control signal? The individual metrics are standard; our contribution is the control-facing protocol and its scope boundaries.

Table 1: Boundary to neighbouring consistency and control-representation work.

Neighbouring thread	Shared concern	Boundary in this paper
ViGMO [24]	Visual distractions and latent consistency	We do not propose a generic encoder-consistency regulariser; our diagnostic places consistency after action-conditioned rollout and adds a discriminability guard.
Bisimulation / Bisim-JEPA [10, 41, 34]	Control-relevant state abstraction	Same-state corrupted and unperturbed views need not share an encoder latent; agreement is required at the predictive-dynamics level.
LeJEPA theory [18]	Latent variables and latent planning	We do not claim identifiability; we test whether learned checkpoints preserve predictive consistency under visual perturbation.
Group-action world models [38]	Action-faithful rollout structure	We test same-action paired corruption consistency, not identity/inverse/composition consistency across action sequences.
Value-aware models [12, 36]	Models should serve downstream control	We make this operational as paired action-conditioned rollout agreement, not value equivalence or value prediction.
V-JEPA family [4, 3, 16]	Latent video prediction and noisy-environment probes	Representation recovery and closed-loop control stress different properties; our probe is control-facing and eval-only.

3 Action-Conditioned Predictive Consistency

This section defines action-conditioned predictive consistency and its discriminability countercondition. Section 3.2 then separates the main descriptive diagnostics from paired ACPC probes.

3.1 Setup and definitions

Let o_t be an unperturbed observation and $\tilde{o}_t = \tau(o_t)$ a visually corrupted view of the *same* underlying state (τ is additive Gaussian pixel noise here, but the definitions are corruption-agnostic). Let h_t and $\tilde{h}_t$ be the corresponding observation–action histories, E_θ the encoder, and F_θ the action-conditioned latent predictor. Fix an action sequence $\mathbf{a} = a_{t:t+H-1}$ that is *identical* on both branches, and write

$$z_t = E_\theta(h_t), \quad \tilde{z}_t = E_\theta(\tilde{h}_t), \qquad \hat{z}_{t+k} = F_\theta^k(z_t, \mathbf{a}_{0:k-1}), \quad \hat{\tilde{z}}_{t+k} = F_\theta^k(\tilde{z}_t, \mathbf{a}_{0:k-1}). \tag{2}$$

Let Π denote a *task-relevant predictive readout* — the full latent, a transition delta, goal-distance or cost features, or a learned projection — and let d be a distance on its range.

Action-conditioned predictive consistency. For a same-state perturbation pair under the shared action sequence,

$$\mathrm{ACPC}_H(o_t, \tilde{o}_t, \mathbf{a}) = \sum_{k=1}^{H} \alpha_k \, d\big(\Pi(\hat{z}_{t+k}), \, \Pi(\hat{\tilde{z}}_{t+k})\big), \qquad \alpha_k \geq 0. \tag{3}$$

Robustness is the requirement that ACPC_H be *small* — crucially *not* that the encoder distance $d(z_t, \tilde{z}_t)$ be small. Encoder closeness is neither necessary (a large but task-irrelevant encoder shift can wash out through F_θ and Π) nor sufficient (a small encoder shift can still change the predicted future enough to flip the planned action).

Action-relevant discriminability (countercondition). Consistency must be *selective*. Let $Y^H(s, \mathbf{a})$ denote a task-relevant future readout that is external to the learned predictor — simulator state, cost-to-go, inverse-dynamics label, contact/keyframe tag, or another diagnostic proxy available in the dataset. For two genuinely different states s_i, s_j (with latents z_i, z_j) that admit an action sequence under which this external future readout diverges,

$$\Delta^H_{\text{dyn}}(s_i, s_j, \mathbf{a}) = d_Y\big(Y^H(s_i, \mathbf{a}),\, Y^H(s_j, \mathbf{a})\big) > m, \tag{4}$$

the representation and predictions must keep them separable,

$$d\big(\Psi(z_i),\, \Psi(z_j)\big) > m', \tag{5}$$

where Ψ is a discriminability readout (latent, predicted delta, inverse-dynamics feature, cost feature, or rollout embedding) and $m, m' > 0$ are margins. Equations (3)–(5) make the central tension precise: it is not invariance versus resolution in the abstract, but

predictive consistency high for same-state visual-perturbation pairs; predictive discriminability high for state/action/transition-distinct pairs.

A method that drove $\text{ACPC}_H \to 0$ by collapsing the latent would violate (5), which is why the two conditions are stated together.

3.2 Operational consistency diagnostics

Status. Descriptive quantities support the main cross-checkpoint analysis; paired ACPC quantities define method-facing probes. The paper-facing paired result is the Gaussian-noise ACPC basin diagnostic on all 36 LeWM checkpoints (Section 5.4) with a full PLDM replication in Appendix E. Appendix G reports broader ACPC/PCC/CRA/MAF/ADM/SPRR probes under the auxiliary pixels+goal stress condition as exploratory diagnostics. ϵ below is the constant from Equation (7).

Encoder perturbation difference (EPD), descriptive.
EPD $= d(E(o), E(\tilde{o}))$. This is the Layer-1 encoder shift of Section 4.3, retained only as a *descriptive* quantity: a large EPD is not in itself bad and a small EPD is not in itself good. It reports whether a perturbation enters the latent, not whether it matters for control.

One-step consistency (ACPC-1), paired probe.
$\text{ACPC}_1 = d\big(F(E(o), a), F(E(\tilde{o}), a)\big)$, optionally normalised by a natural transition magnitude, $\text{ACPC}_1/(d(z_t, z_{t+1}) + \epsilon)$, so that it is comparable across checkpoints.

Multi-step consistency (ACPC-H), paired probe.
Equation (3) with $H \in \{4, 8\}$. This is the principled, *paired* version of the existing multi-step rollout drift (`predictor_rollout_T8_l2`): it measures disagreement between unperturbed and corrupted rollouts under the *same* action sequence, not generic rollout drift.

ACPC basin radii, reported diagnostic.
For each state we evaluate the original view plus Gaussian-noised views at std $\in \{0.01, \ldots, 0.08\}$. In the main reported basin diagnostic, Π is the identity on the model's inference/cost latent sequence, and d is Euclidean L2 on those latent tokens. We report an encoder radius R_E (median pairwise encoder-history L2 spread, normalised by the original-view NN L2 scale), a prediction radius R_F (median pairwise $H = 8$ rollout-latent L2 spread, normalised by the clean transition L2 reference over the same horizon), and a contraction ratio R_F/R_E. This is the dense paired diagnostic in `assets/paper1_data/acpc_basin_diagnostics.json` and `assets/paper1_data/acpc_basin_diagnostics_pldm.json`; alternative readouts are generalisations used only in Appendix G or future methods.

Predictive cost consistency (PCC), downstream readout.

$\mathrm{PCC} = \left|J(F^H(E(o), \mathbf{a}), g) - J(F^H(E(\tilde{o}), \mathbf{a}), g)\right|$ for a cost/goal readout J and goal g. PCC tests whether predictive mismatch changes the planning cost; it is a downstream readout, not the central definition.

Candidate-ranking agreement (CRA), downstream readout.

For a candidate set $\mathcal{A} = \{\mathbf{a}^1, \ldots, \mathbf{a}^K\}$ held identical across branches, the Spearman/Kendall agreement of the unperturbed and corrupted cost vectors. We treat CRA as a downstream readout, require the candidate set to be shared, and do not call it planning equivalence.

Margin-conditioned action flip (MAF), downstream readout.

$\mathrm{MAF} = \mathbf{1}\left[u^*_{\mathrm{orig}} \neq u^*_{\mathrm{noise}},\ C_{(2)} - C_{(1)} > \delta\right]$: a flip of the selected action counts only when the unperturbed top-1/top-2 cost margin exceeds δ, since a flip inside the noise margin need not be a failure.

Action-relevant discriminability margin (ADM), paired probe.

Over action/transition-distinct pairs P_{disc} (differing inverse-dynamics labels, large original-view transition magnitude, contact/keyframe transitions, or large cost-to-go change), $\mathrm{ADM} = \mathrm{median}_{(i,j)\in P_{\mathrm{disc}}}\, d(\Psi_i, \Psi_j)$. A consistency method should reduce ACPC_H without materially reducing ADM.

Selective predictive robustness ratio (SPRR), summary probe.

A single summary of action-distinct discriminability relative to original/corrupted predictive inconsistency,

$$\mathrm{SPRR} = \frac{\mathrm{median}_{(i,j)\in P_{\mathrm{disc}}}\, d(\Psi_i, \Psi_j)}{\mathrm{median}_{P_{\mathrm{pert}}}\, \mathrm{ACPC}_H + \epsilon}.$$

We use it descriptively and do not claim it predicts success.

These diagnostics localise mechanisms and define method targets; consistent with the negative results in Section 5.6, we do not claim that any of them *predict* robustness.

4 Background and Diagnostic Framework

4.1 LeWorldModel baseline

LeWM [23] is an end-to-end JEPA world model trained with two terms only:

$$\mathcal{L}_{\mathrm{LeWM}} \;=\; \mathcal{L}_{\mathrm{pred}} \;+\; \lambda \cdot \mathcal{L}_{\mathrm{SIGReg}}\,, \tag{6}$$

where $\mathcal{L}_{\mathrm{pred}}$ is the latent-space mean-squared error (MSE) between predicted and target representations. SIGReg projects each latent onto M unit-norm random directions, computes the Epps–Pulley empirical-characteristic-function distance [7] between each projection and $\mathcal{N}(0,1)$, and aggregates with Gauss-window weights, preventing collapse without explicit BatchNorm. (The Cramér–Wold theorem motivates the construction: equality of high-dimensional distributions reduces to equality of all one-dimensional projections of their characteristic functions.) Inference uses the Cross-Entropy Method (CEM) [20] for latent-space model predictive control (MPC) [8].

4.2 Input-side noise augmentation

We add per-frame Gaussian noise to the LeWM input pipeline via `utils.py::AddNormalizedGaussianNoise` (Section A). Each frame is independently noised: Bernoulli(p) decides whether the frame is corrupted, and if so the standard deviation is drawn from $\mathrm{Uniform}(0, \mathrm{std\_max})$. The training transform is applied to the whole `pixels` sequence, so the predictive target latent is also encoded from a noised future frame; this is a full-sequence augmentation pressure, *not* an original-target denoising objective. Appendix H shows that simply switching to an original-future target does not recover noisy closed-loop control, so the main sweep retains the full-sequence branch. We fix $p = 1.0$ and sweep `std_max` $\in \{0.01, \ldots, 0.08\}$ (eight levels). Our primary noised evaluation applies Gaussian noise only to the observation pixels while leaving the goal image unperturbed; pixels+goal evaluation is retained as a stronger full-visual-stream stress test.

4.3 Five-layer diagnostic framework

The framework decomposes the JEPA world-model control forward pass — pixels $\to$ encoder f $\to$ latent z $\to$ predictor g $\to$ cost surface $\to$ planned actions (optimised with CEM) — into five sequential stages and instruments each transition with one or more metrics. (In this diagnostic context we write f and g for the encoder E_θ and action-conditioned predictor F_θ of Section 3.) Layers 1–2 characterise the encoder output; Layer 3 measures how the predictor amplifies an encoder shift; Layer 4 injects noise directly into the latent to isolate predictor and cost contributions; Layer 5 quantifies planning-relevant information retained in the latent. Most individual metrics already have a literature home (cited inline below). Our additions are scale-normalised ratios that compare perturbation effects with the unperturbed local nearest-neighbour scale. We use nearest-neighbour distance as a local latent scale primitive in the spirit of k-nearest-neighbour (kNN) out-of-distribution detection [31], but the ratios themselves are introduced here.

Relation to action-conditioned predictive consistency. The five layers decompose input-side noise augmentation into encoder shift, encoder geometry, predictor sensitivity, latent-noise response, and task resolution. Layers 1–2 describe whether a perturbation enters the latent; Layer 3 is closest to predictive consistency through fragility ratio and rollout drift; Layer 5 measures the discriminability side through transition resolution and inverse-dynamics probes. We treat encoder closeness as descriptive only, because robustness is tested after the action-conditioned predictor. Figure 1 illustrates the boundary, and Table 11 summarises the task-level reading.

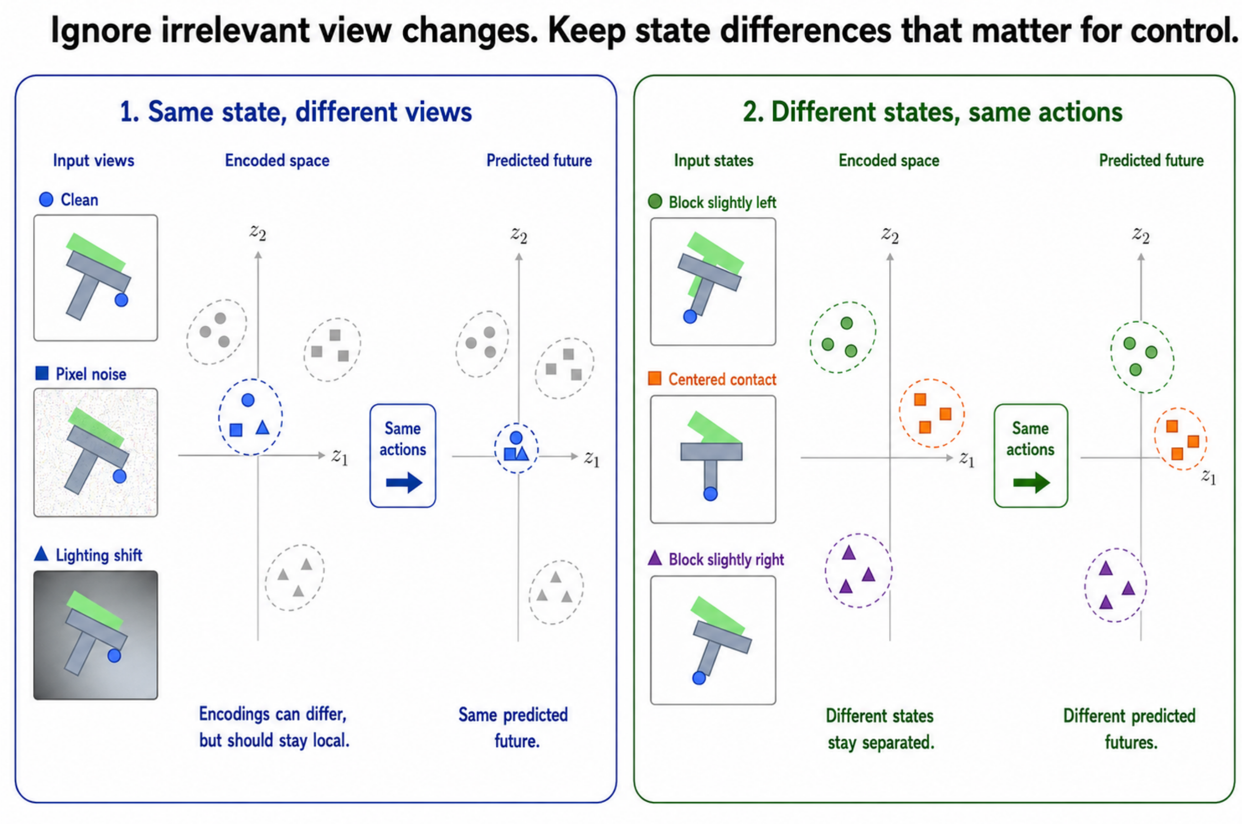


Figure 1: Conceptual reading of the selective-consistency tension. Same-state nuisance variants should induce consistent action-conditioned predictions, while states that differ in transition, cost, or optimal action must remain resolvable. The bottom axis is a qualitative task read backed by the sweep and diagnostics, not a literal ordering of required state resolution. PLDM serves as a second-family task-level replication; the exact diagnostic mechanism remains method-specific.

Minimal notation. Let $z_i = f(x_i)$ and $\tilde{z}_i = f(\tilde{x}_i)$ be unperturbed and noised latents for the same token. The distance, local nearest-neighbour (NN) scale, and three introduced ratios are defined together as

$$\begin{aligned} d_{\cos}(a,b) &= 1 - \frac{a^\top b}{\|a\|\,\|b\|}, \\ d_i^{\mathrm{NN}} &= \min_{j\neq i} d_{\cos}(z_i, z_j), \\ r_i^{\mathrm{enc}} &= \frac{d_{\cos}(z_i,\tilde{z}_i)}{d_i^{\mathrm{NN}}+\epsilon}, \\ r_i^{\mathrm{pred}} &= \frac{d_{\cos}\big(g(z)_i, g(\tilde{z})_i\big)}{d_i^{\mathrm{NN}}+\epsilon}, \\ R_d &= \frac{\mathrm{median}_t\ d(z_t, z_{t+1})}{\mathrm{median}_{(t,t')\in\mathcal{F}}\ d(z_t, z_{t'})+\epsilon}. \end{aligned} \tag{7}$$

Here r_i^{enc} is the encoder-shift ratio, r_i^{pred} is the fragility ratio, and R_d is the transition-resolution ratio for either cosine or L2 distance d. $\mathcal{F}$ denotes random far pairs from different temporal locations, and $\epsilon = 10^{-8}$ is a numerical-stability constant. Released JSON artifacts keep the implementation keys (`noise_to_nn_cos_ratio`, `predictor_target_to_nn_cos_ratio_at_max_std`, and `transition_resolution_ratio_{cos,l2}`), but the main text uses the shorter prose names above.

Layer 1 — Encoder shift. The magnitude and direction of latent displacement under input noise. Metrics: raw angular/L2 shift, angular gain per unit noise, and the encoder-shift ratio in Equation (7).

Layer 2 — Encoder geometry. The global structure of the encoded latent space. Metrics: local neighbourhood scale, effective rank [27], and Centered Kernel Alignment (CKA) [19] between unperturbed-input and noisy-input latents at the diagnostic's maximum injection std.

Layer 3 — Predictor sensitivity. How the predictor amplifies the encoder shift. Metrics: the *fragility ratio* in Equation (7) and multi-step rollout L2 drift at horizon T.

Layer 4 — Latent-noise response. Noise injected directly into the latent z (bypassing the encoder) isolates predictor and cost contributions. Metrics: the slope of the planner's cost surface under latent perturbations and the latent robust radius, defined as the largest perturbation that keeps the cost ranking stable.

Layer 5 — Task resolution. The control-relevant information retained in the latent. Metrics: the L2/cosine transition-resolution ratio R_d in Equation (7) and the inverse-dynamics linear-probe R^2, a controllability proxy in the spirit of linear-probe analysis [1].

Reading guidance. Layers 1, 2, 4, and 5 are descriptive. Layer 3 supplies the two full-coverage cross-checkpoint metrics used in Section 5.6: fragility ratio and multi-step rollout drift.

4.4 Cross-checkpoint validation protocol

For each task we compute Spearman ρ across the 9 LeWM checkpoints (base + `std_max` $\in \{0.01, \ldots, 0.08\}$) and partial Spearman conditioned on `std_max`. The partial step removes the sweep-level training-noise trend, asking whether a diagnostic retains residual checkpoint-quality signal. We report 95% percentile bootstrap intervals ($B = 1000$, seed $= 42$); the bootstrap artifact is `assets/paper1_data/partial_corr_bootstrap_20260523.json`. PLDM replication is reported in Section E.

5 Experiments

5.1 Setup

Tasks. PushT (2D pushing), TwoRoom (2D navigation), Reacher (2D arm), and Cube (3D manipulation) form a heterogeneous stress panel spanning different nuisance/discriminability regimes; it is not intended as an average benchmark over all control tasks.

Baselines. LeWM-base (no noise) and LeWM+noise (8-level sweep).

Checkpoints. We use 36 LeWM checkpoints: for each task, one no-noise baseline plus eight `std_max` values.

Evaluation protocol. All 36 LeWM checkpoints use $n = 3$ evaluation seeds (42/43/44) and `num_eval = 100` trajectories per seed. These are evaluation seeds, not independent training seeds: each `std_max` cell corresponds to one trained epoch-10 checkpoint in the released manifest. Tables 2, 3, 4 report across-seed population mean ± std from `assets/paper1_data/canonical_evals_20260517.json`. Some artifacts use *clean* as the condition name; the claims use *unperturbed* or *original view*. Diagnostic source-of-truth is `assets/paper1_data/canonical_diagnostics_20260517.json`.

Hardware. Single NVIDIA H800 (80 GB).

5.2 JEPA control fragility under visual corruptions

Table 2 reports LeWM-base success rates under unperturbed and noised evaluation (mean ± std across 3 seeds × 100 evaluations).

Table 2: LeWM-base under test-time visual corruptions (mean ± std; 3 seeds × 100 evaluations).

Task	unperturbed	pixels 0.05	pixels 0.08	px+goal 0.08	unperturbed → pixels 0.08 drop
TwoRoom	94.00 ± 3.56	72.33 ± 5.79	65.67 ± 1.25	50.00 ± 1.41	-28.33
PushT	86.33 ± 2.36	17.00 ± 1.41	4.33 ± 1.25	4.67 ± 2.05	-82.00
Reacher	58.67 ± 1.25	33.67 ± 2.87	18.33 ± 2.05	15.00 ± 2.16	-40.33
Cube	66.67 ± 2.62	48.67 ± 6.85	47.00 ± 6.38	46.33 ± 3.68	-19.67

LeWM-base is strong on unperturbed images (especially TwoRoom and PushT), but observation noise alone already produces large closed-loop drops: PushT loses 82.00 pts at pixels 0.08, Reacher 40.33 pts, TwoRoom 28.33 pts, and Cube 19.67 pts. The pixels+goal column shows the stronger full-visual-stream stress case; it further degrades TwoRoom and Reacher, but is not the primary robustness endpoint because the goal image is also perturbed. The drop pattern across tasks remains informative: Cube degrades least, while PushT degrades most sharply, consistent with contact-heavy continuous control being most sensitive to visual precision.

The same direction of failure is visible on PLDM but with task-dependent strength. PLDM trained without noise augmentation drops sharply on PushT (75.33% ± 3.68 unperturbed to 18.33% ± 2.05 at pixels 0.08, −57.00 pt) and TwoRoom (96.67% ± 1.70 to 67.00% ± 2.16, −29.67 pt), while Reacher and Cube have much smaller no-noise-training gaps (−2.33 and −4.33 pt; Table 15). The noise-training signatures still carry over: TwoRoom recovers into a high-noise plateau, PushT has a large `std_max` ≈ 0.03 recovery, and Cube responds weakly (Section E).

5.3 Noise augmentation closes the gap as a coarse pressure

Tables 3 and 4 report the complete 8-level sweep across tasks. All cells are success rate ± across-seed std (in pts), aggregated under the unified $n = 3$ seeds (42/43/44) × 100 trajectories protocol — 300 trajectories per cell.

Table 3: LeWM+noise sweep, unperturbed success rate (%; released condition-name: `clean`).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	94.00 ± 3.56	86.33 ± 2.36	58.67 ± 1.25	66.67 ± 2.62
0.01	93.67 ± 3.30	88.00 ± 3.74	61.67 ± 2.49	69.33 ± 0.47
0.02	95.00 ± 2.83	88.33 ± 2.62	85.67 ± 2.49	60.00 ± 1.63
0.03	96.33 ± 3.30	89.67 ± 1.70	78.67 ± 1.25	65.00 ± 1.63
0.04	96.33 ± 2.05	89.33 ± 2.05	84.00 ± 2.94	69.00 ± 3.74
0.05	96.00 ± 2.83	80.67 ± 4.78	70.00 ± 2.16	59.33 ± 0.94
0.06	96.67 ± 2.05	89.33 ± 2.05	86.00 ± 2.94	66.67 ± 2.05
0.07	96.00 ± 1.63	85.67 ± 3.09	83.67 ± 3.30	67.67 ± 0.94
0.08	98.33 ± 0.47	88.33 ± 2.87	84.00 ± 0.82	62.33 ± 1.25

Table 4: LeWM+noise sweep, pixels 0.08 success rate with unperturbed goal (%).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	65.67 ± 1.25	4.33 ± 1.25	18.33 ± 2.05	47.00 ± 6.38
0.01	94.00 ± 2.83	51.67 ± 5.44	45.67 ± 3.86	49.00 ± 2.45
0.02	94.33 ± 3.09	73.67 ± 4.99	83.67 ± 2.49	58.67 ± 3.30
0.03	97.00 ± 2.94	82.00 ± 6.48	78.67 ± 2.87	68.33 ± 2.49
0.04	96.67 ± 2.05	86.67 ± 2.87	82.00 ± 0.82	65.00 ± 2.45
0.05	95.33 ± 3.30	78.33 ± 5.91	70.00 ± 2.94	60.00 ± 0.82
0.06	96.33 ± 2.05	87.67 ± 3.30	81.33 ± 1.70	66.67 ± 1.70
0.07	96.67 ± 1.25	84.00 ± 3.74	84.67 ± 5.19	67.33 ± 2.05
0.08	97.67 ± 0.94	89.00 ± 4.32	82.00 ± 1.41	62.00 ± 1.41

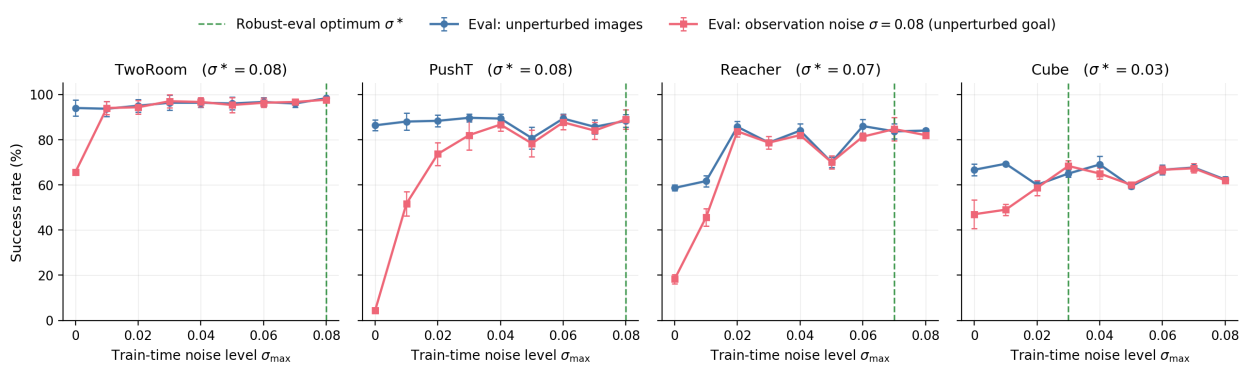


Figure 2: Noise-training sweep across all four tasks. Blue: success rate under unperturbed evaluation. Red: success rate under observation-only Gaussian noise $\sigma = 0.08$ with an unperturbed goal. Green dashed: each task's robust-eval optimum σ^* (the train-time `std_max` that maximises corrupted-eval success). The optimum σ^* varies across tasks, consistent with a coarse global scalar pressure with broad, task-dependent plateaus rather than a single jointly optimal level.

The sweep has two readings. First, input-side noise is effective: PushT recovers from 4.33% to 89.00% at pixels 0.08, and Reacher from 18.33% to 84.67%. Second, the pressure is coarse and task-dependent. TwoRoom favours heavy noise, PushT's unperturbed and noisy-eval optima dissociate (0.03 vs 0.08), Reacher has a broad plateau, and Cube responds weakly. Several optima are plateau regions rather than isolated points, so the claim is not that exact per-task tuning is always necessary; it is that one scalar perturbation strength cannot express the selective-consistency tradeoff by itself.

The behavioural evidence for this selective-consistency tension is summarised by the sweep tables and Figure 2. Before turning to the broader five-layer diagnostic suite, we first report a paired Gaussian-noise ACPC basin diagnostic that directly checks whether the same-state noisy views occupy a smaller action-conditioned predictive region after noise training.

5.4 Paired Gaussian-noise ACPC basin diagnostic

We compute an eval-only ACPC basin diagnostic on the same 36 LeWM epoch-10 checkpoints used in the sweep. For each checkpoint we sample 100 fixed dataset windows and create eight paired same-state views using Gaussian pixel noise with std $\in \{0.01, \ldots, 0.08\}$, matching the training sweep's corruption family. Each original/noised branch is encoded and then rolled out under the same recorded action sequence for horizon $H = 8$. The reported prediction readout is the identity on the resulting inference/cost latent rollout sequence, with L2 distance over rollout tokens. The released source is `assets/paper1_data/acpc_basin_diagnostics.json`, generated by `tools/paper1_acpc_basin.py`. The diagnostic intentionally excludes blur/resize so that the ACPC evidence is not confounded by train/eval corruption-family mismatch.

For each checkpoint, let R_E be the median pairwise spread among the original and noised encoder-history views, normalised by the original-view local NN L2 scale, and let R_F be the median pairwise spread among their $H = 8$ predicted rollout latents, normalised by the clean transition L2 reference over the same horizon. Lower R_F means the same-state perturbation views induce a tighter action-conditioned predictive basin; R_F/R_E is a descriptive contraction ratio rather than a success predictor.

Appendix G provides an exploratory downstream check under the same shared-candidate action protocol: predictive cost consistency, candidate-ranking agreement, and margin-conditioned action flips move in the same direction at the fragile endpoints. We keep these readouts as mechanism-localisation evidence rather than robustness predictors.

Table 5: Gaussian-noise ACPC basin diagnostic on LeWM: no-noise baseline vs. each task's pixels 0.08 robust-best checkpoint. R_E is normalised encoder-view spread; R_F is normalised action-conditioned rollout-view spread under the same action sequence. The diagnostic uses Gaussian-noise views of the observation history (std 0.01–0.08) while leaving the goal unperturbed, matching the primary unperturbed-goal evaluation.

Task	checkpoint	pixels 0.08	drop	R_E	R_F
TwoRoom	base	65.67	28.33	3.538	0.785
TwoRoom	noise 0.08	97.67	0.67	0.235	0.028
PushT	base	4.33	82.00	1.727	1.543
PushT	noise 0.08	89.00	−0.67	0.137	0.088
Reacher	base	18.33	40.33	4.238	1.012
Reacher	noise 0.07	84.67	−1.00	0.135	0.027
Cube	base	47.00	19.67	1.343	0.957
Cube	noise 0.03	68.33	−3.33	0.238	0.115

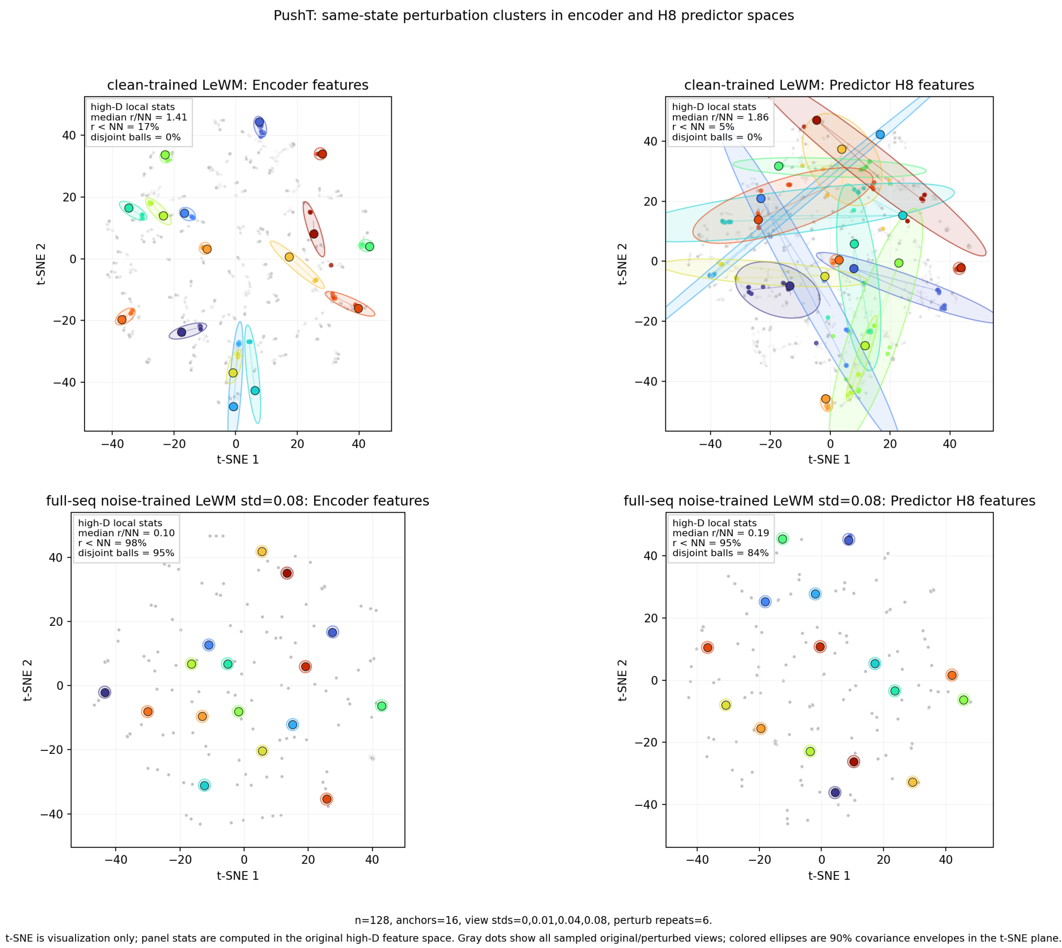


Figure 3: Qualitative PushT view of same-state perturbation samples corresponding to Table 5. Each coloured anchor denotes one dataset state; coloured local samples are repeated Gaussian perturbation views of that same state, and grey points show the sampled background cloud. Top: clean-trained LeWM; bottom: full-sequence noise-trained LeWM at `std_max` $= 0.08$. Left panels use encoder-history features; right panels use $H = 8$ predictor-rollout features. The t-SNE plane and 90% 2-D covariance envelopes are visualization only; inset statistics (*median r/NN*, $r <$ NN, and *disjoint balls*) are computed in the original high-dimensional feature space. This is a representative mechanism view; the multi-task quantitative evidence remains Table 5.

The reading is deliberately narrow. The diagnostic supports the ACPC framing because robust noise-trained checkpoints have much smaller action-conditioned prediction radii under same-family visual perturbations: e.g., PushT reduces R_F from 1.543 to 0.088, and TwoRoom from 0.785 to 0.028. Figure 3 provides a representative PushT visualization of this contraction, but it should not be read as a standalone proof or a high-dimensional basin boundary; Appendix A.6 adds a projection-free local-atlas check of the same data. The result does *not* support the stronger universal claim that encoder representations remain widely separated while predictions alone collapse them. In this Gaussian-noise setting, robust checkpoints usually shrink both the encoder radius and the predictive radius, with the predictor-space radius remaining the control-facing quantity. This is why we use the basin diagnostic to refine the mechanism claim, not to replace the behavioural corruption evaluation.

The broader representation-level evidence appears next in Table 6.

5.5 Diagnostic analysis: why global noise is only a coarse pressure

Where the per-task sweep evidence in Section 5.3 is the behavioural face of selective consistency, the diagnostics in this subsection are its representational face: which latent properties move under noise training, and whether the movement is bounded by each task's discriminability requirements. Table 6 compares core diagnostic metrics on LeWM-base versus the per-task representative noise-trained diagnostic checkpoint.

Table 6: Representation diagnostics: LeWM-base vs. a representative noise-trained diagnostic checkpoint per task. The σ picks (0.08 / 0.02 / 0.06 / 0.01) are the checkpoints on which the diagnostic suite was originally run. PushT's unperturbed point-best is `std_max` = 0.03, within ±2 pt of the diagnosed `std_max` = 0.02 checkpoint, so we keep the original diagnostic row.

Metric	TwoRoom base	TwoRoom noise (0.08)	PushT base	PushT noise (0.02)	Reacher base	Reacher noise (0.06)	Cube base	Cube noise (0.01)
`clean_nn_cos_dist_median`	0.0449	0.0321	0.2360	0.2477	0.0633	0.0676	0.1856	0.1879
`clean_effective_rank`	47.60	37.69	76.42	77.41	61.04	65.92	73.25	71.83
`transition_resolution_ratio_l2`	0.7216	0.6621	0.3015	0.2989	0.3704	0.3791	0.4847	0.4629
`transition_resolution_ratio_cos`	0.5538	0.4610	0.0868	0.0867	0.1351	0.1399	0.2347	0.2168
`id_probe_r2`	0.2889	0.1419	0.7739	0.7690	0.1621	0.1729	0.6657	0.6720
`action_mean_pred_shift_norm`	0.5329	0.4843	0.1283	0.1305	0.2518	0.2585	0.2364	0.2320
`predictor_rollout_T8_l2`	18.62	0.66	18.65	6.00	15.17	0.44	20.20	19.25

Notes. (i) All representative diagnostics are taken from the corresponding checkpoint's per-tool JSON outputs under the diagnostics directory (max-std = 0.1, history-only noise); (ii) the near-collapse of multi-step rollout drift at the heavier-noise representatives (TwoRoom 18.62 → 0.66 at `std_max` = 0.08; Reacher 15.17 → 0.44 at `std_max` = 0.06) is not a recording error: sufficient training noise collapses the multi-step predictor drift while leaving the single-step task-resolution metrics comparatively flat — precisely the dissociation that motivates the analysis in Section 5.6. The low-noise representatives reduce drift far less (PushT 18.65 → 6.00 at `std_max` = 0.02; Cube 20.20 → 19.25 at `std_max` = 0.01).

Mechanistic reading.

- **TwoRoom.** Low-dimensional, discrete, visually redundant. Compressing the representation (effective rank 47.6 → 37.7) is acceptable and even beneficial: a more compact latent space is easier to plan in.
- **PushT.** Continuous contact requires fine-grained pose resolution. Noise training largely respects this bound: at the representative checkpoint (`std_max` = 0.02) the effective rank and inverse-dynamics probe stay flat (76.4 → 77.4; 0.774 → 0.769), and the L2 transition-resolution metric declines only mildly and monotonically toward heavier noise (0.3015 at base, 0.2989 at 0.02, 0.2789 at 0.08; per-checkpoint diagnostics under `eval_results/diagnostics/`). The catastrophic resolution erasure on PushT appears under error-based reweighting (Section D), not under input-side noise.
- **Reacher.** Low-dimensional continuous reaching does not show a resolution collapse in Table 6: effective rank, transition resolution, and the controllability probe remain flat or slightly improve. The notable change is the large reduction in multi-step predictor drift (15.17 → 0.44), matching the later finding that Reacher's residual corruption-drop signal lives in the rollout metric rather than in the single-step fragility ratio.
- **Cube.** Cube is moderately resolution-demanding but less fragile than PushT. The representative checkpoint leaves rank, transition resolution, controllability probe, and rollout drift almost

unchanged, consistent with Cube's smaller visual-corruption cliff and the moderate residual signal of multi-step drift in Table 8.

- **Predictor-rollout caveat.** A drop in multi-step predictor rollout drift is not unambiguously good news. It can also mean the latent has become more *predictable* without being more *controllable*: predictor stability bought by sacrificing resolution.

5.6 Cross-checkpoint correlation analysis

Table 7 reports task-specific cross-checkpoint correlations on the LeWM $n = 9$ sweep using the unified 3-seed $\times$ 100 evaluation.

We analyse two single-value-per-checkpoint diagnostic metrics that have full coverage on all 9 checkpoints per task: the fragility ratio, defined in Section 4.3 as the single-step predictor target shift normalised by the nearest-neighbour cosine distance at the diagnostic's max-std injection, and the corresponding multi-step rollout L2 drift. Both are released in `assets/paper1_data/canonical_diagnostics_20260517.json`, derived from each checkpoint's predictor-sensitivity diagnostic JSON, and are pure checkpoint-level quantities (independent of the evaluation protocol).

Table 7: LeWM $n = 9$ sweep: per-task Pearson r / Spearman ρ vs corruption drop (unperturbed $-$ pixels 0.08, unperturbed goal). Eval values come from the unified 3-seed $\times$ 100 protocol.

Metric	TwoRoom (r/ρ)	PushT (r/ρ)	Reacher (r/ρ)	Cube (r/ρ)
`predictor_target_to_nn_cos_ratio_at_max_std`	+0.92/+0.20	−0.55/−0.80	+0.98/+0.55	+0.39/+0.27
`predictor_rollout_T8_l2_at_max_std`	+0.81/+0.31	+0.98/+0.93	+0.99/+0.58	+0.91/+0.48

Reading. Unconditional correlations are large because both diagnostic metrics and the corruption drop are driven by the same upstream variable — `std_max`. The relevant test is partial correlation conditioned on `std_max` (Table 8).

Table 8: Partial Spearman ρ(metric, corruption drop | `std_max`), $n = 9$ per task.

Metric	TwoRoom	PushT	Reacher	Cube
`predictor_target_to_nn_cos_ratio_at_max_std`	−0.03	+0.19	+0.27	+0.12
`predictor_rollout_T8_l2_at_max_std`	0.00	−0.01	+0.37	+0.23

After removing the monotone sweep trend associated with `std_max`, the **fragility metric carries little stable information about the size of the corruption drop** on any task; all bootstrap intervals for the partial correlations include zero. The **multi-step predictor drift** retains only weak residual signal under the unperturbed-goal pixels 0.08 endpoint (Reacher +0.37, Cube +0.23, both with wide intervals). This weakens the older pixels+goal-stress reading and is the reason we treat the cross-checkpoint diagnostic as mechanism localisation, not as a robustness predictor.

Table 9: Spearman ρ (unconditional) and partial Spearman ρ conditioned on `std_max`, for the fragility ratio on the PushT $n = 9$ LeWM sweep under the unified 3-seed $\times$ 100 eval. CIs are the percentile 95% interval over $B = 1000$ bootstrap resamples (with replacement, seed = 42; see Section 4.4). The `std_max`-only top four rows are univariate sweep diagnostics with no metric/outcome pairing, so no CI is reported.

Quantity	Spearman ρ	95% bootstrap CI
ρ(`std_max`, metric)	+0.83	—
ρ(`std_max`, unperturbed)	0.00	—
ρ(`std_max`, pixels 0.08)	+0.88	—
ρ(`std_max`, corruption drop)	−0.98	—
ρ(metric, unperturbed) unconditional	−0.33	[−0.95, +0.62]
ρ(metric, unperturbed) | `std_max` partial	−0.59	[−0.97, −0.10]
ρ(metric, pixels 0.08) unconditional	+0.60	[−0.11, +1.00]
ρ(metric, pixels 0.08) | `std_max` partial	−0.53	[−0.84, +0.00]
ρ(metric, corruption drop) unconditional	−0.80	[−1.00, −0.23]
ρ(metric, corruption drop) | `std_max` partial	+0.19	[−0.00, +0.70]

PushT $n = 9$ detailed view (fragility ratio). Interpretation:

1. **After removing the monotone trend associated with `std_max`, the metric still carries a residual checkpoint-quality signal.** Partial ρ on unperturbed evaluation is −0.59 and on pixels 0.08 is −0.53 — both negative. On the $n = 9$ PushT sweep, lower fragility ratio aligns with better unperturbed *and* better noisy success beyond what the sweep-level `std_max` trend already explains.
2. **The metric does NOT predict the unperturbed-to-corrupted gap.** The unconditional $\rho(\text{metric}, \text{drop}) = -0.80$ looks impressive, but partialling out `std_max` reduces it to +0.19 (95% bootstrap CI [−0.00, +0.70], including 0). The drop's strong correlation with the metric is a mediated effect: `std_max` drives both the metric ($\rho = +0.83$) and the drop ($\rho = -0.98$). After the `std_max` trend is removed, the metric tells you little stable information about how much the *gap* between unperturbed and noisy performance will be.
3. **Unperturbed sign reversal.** $\rho(\text{metric}, \text{unperturbed})$ is only −0.33 unconditionally — unperturbed success rate is roughly flat across the PushT sweep under the 3-seed protocol (range 80.67–89.67), so `std_max` barely moves unperturbed performance. The partial correlation *strengthens* to −0.59 once the sweep-level `std_max` trend is removed, exactly because the residual signal is what the metric tracks.
4. **Practical reading.** The toolkit is useful for mechanism localisation and checkpoint triage after controlling for the sweep-level `std_max` effect. It is *not* a substitute for training and evaluating under corruptions when the robustness gap is the quantity of interest.

5.6.1 Unperturbed control fidelity and corruption robustness as separable signals

The partial-correlation analysis above supports the following interpretation:

- The metric is a **residual checkpoint-quality signal beyond the sweep-level `std_max` effect** — a lower fragility ratio means better control on *both* unperturbed and noisy evaluation (partial $\rho = -0.59$ / -0.53 on PushT).
- The metric **does not isolate noise robustness** — its apparent correlation with the gap between unperturbed and noisy success is mediated by `std_max` (partial $\rho = +0.19$, wide CI including zero).

The PushT scatter (Figure 4) makes both halves visible. Panel (a) plots metric vs unperturbed success (unconditional Spearman $\rho = -0.33$; the residual trend after conditioning on `std_max` is what the partial correlation captures). Panel (b) plots metric vs corruption drop (unconditional $\rho = -0.80$), but the colour-bar (encoding `std_max`) reveals the structure: low-`std_max` checkpoints (light blue) live at high drop, high-`std_max` checkpoints (dark blue) live at low drop, and the metric tracks `std_max` itself. After the `std_max` trend is removed, the metric does not robustly separate small-drop from large-drop checkpoints.

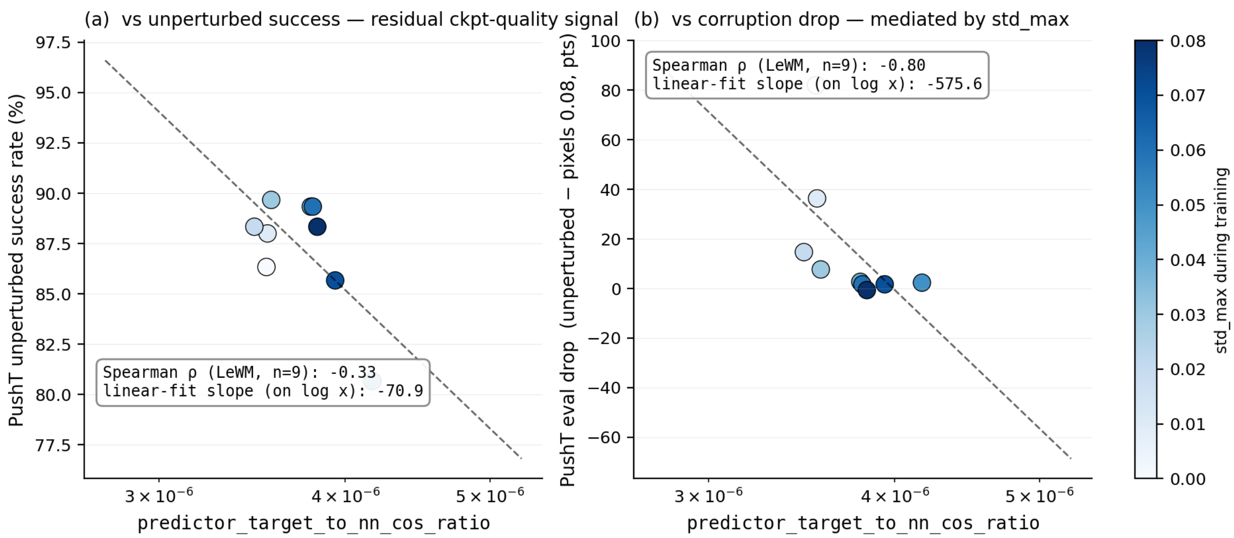


Figure 4: PushT $n = 9$ LeWM sweep: the fragility ratio is a residual checkpoint-quality signal for unperturbed success (a); the apparent corruption-drop correlation in (b) is mediated by `std_max` (encoded by the colour bar).

5.7 Mechanism attribution: where in the pipeline does noise cause failure?

Section 5.5 reports *what moves* in the representation and Section 5.6 reports *which diagnostics* correlate with eval across checkpoints, but neither by itself localises whether the perturbation enters through the encoder or later predictor/planner stages. We therefore use latent-noise probing as a conservative mechanism-localisation check.

5.7.1 Latent-noise probing: encoder shift is consistently implicated

Directly injecting noise into the latent z (skipping the encoder) decouples encoder contributions from predictor + cost. Comparing diagnostic signals across two injection points (pixels vs. latent z, both history-only noise at max std):

- **PushT.** Multi-step input-space rollout drift has unconditional $\rho = +0.93$ vs corruption drop (Table 7), driven primarily by `std_max`; after removing the monotone `std_max` trend, the partial ρ collapses to -0.01. The single-step fragility ratio (unconditional $\rho = -0.80$; partial $+0.19$ after removing the same trend) tells the same story. Both encoder–predictor signals are mediated by training noise; once that sweep-level effect is accounted for, neither isolates the corruption drop on PushT.
- **Reacher.** Multi-step rollout drift has unconditional $\rho = +0.58$ and partial $\rho = +0.37$ vs corruption drop (95% bootstrap CI $[-0.35, +0.99]$). Under the unperturbed-goal observation-noise endpoint, this is only a weak residual signal.
- **Cube.** Multi-step rollout drift has unconditional $\rho = +0.48$ and partial $\rho = +0.23$ (95% CI $[-0.44, +0.96]$), so Cube's residual is also weak.

- **TwoRoom.** Partial correlations are finite but near zero after conditioning on `std_max`; saturation still limits what can be inferred from cross-checkpoint ranks.

Table 10 summarises the three-layer attribution.

Table 10: Three-layer attribution by task (LeWM $n = 9$ sweep, unified 3-seed $\times$ 100 eval).

Task	Strongest signal in these probes	Residual after removing the `std_max` trend
TwoRoom	encoder-shift signal (rank-saturated)	n/a
PushT	encoder + single-step predictor (both mediated by `std_max`)	no residual metric isolates corruption drop
Reacher	encoder + multi-step rollout	weak multi-step drift residual signal (partial $\rho = +0.37$)
Cube	encoder	weak residual on multi-step drift (partial $\rho = +0.23$)

Across these probes, the strongest common signal is **encoder shift transduced by the predictor**. A stronger causal claim about planner or cost-surface contributions would require multi-task latent-injection or cost-ranking ablations, which we do not include here. The present evidence supports mechanism localisation: once the encoder's latent-neighbourhood structure is corrupted beyond the NN distance scale, downstream predictor and planner can operate on the wrong neighbourhood.

6 Discussion

6.1 From latent invariance to predictive consistency

The data challenge an implicit community assumption: latent prediction alone does not guarantee robustness to visual noise. But the corrective is *not* to demand that corrupted and unperturbed images map to the same latent. Our results are better read through the predictive-consistency lens of Section 3: encoder-level latent closeness is neither necessary nor sufficient (Section 3.1), so the target is not $z \approx \tilde{z}$ but agreement of the action-conditioned predictions, $F^k(z, \mathbf{a}) \approx F^k(\tilde{z}, \mathbf{a})$, in task-relevant coordinates. The invariance a JEPA encoder acquires is protocol-dependent; when test-time pixel noise is not covered by a compatible closed-loop training signal, the latent neighbourhood structure can shift, the predictor can roll out the wrong future, and the planner can fail. What matters for control is whether same-state perturbations stay predictive-consistent while action-sensitive state differences stay discriminable.

This is compatible with VJEPA [16], which reports noisy-environment experiments where VJEPA-style models filter high-variance nuisance distractors. Representation-recovery probes only need the latent to preserve recoverable state information, whereas closed-loop control demands a representation *and* predictor that support precise action optimisation — exactly the action-conditioned predictions on which we place the consistency requirement.

6.2 Task-dependent manifestation and scope

The selective-consistency demand — contract nuisance perturbations of the same state, preserve action-relevant transitions — has the following four-task signature in this paper. We use two qualitative task-property labels below: "nuisance-contraction demand" is the room to contract irrelevant visual variation, and "control-discriminability demand" is the action-relevant information that must survive.

- **TwoRoom.** Background wall colour / texture is largely redundant for the navigation objective; reducing sensitivity to it does not impair success in our sweep. This does not mean that all positions can collapse: room, doorway, and goal-topology distinctions must remain separable.
- **PushT.** The pixel changes during T-block / arm contact carry contact-relevant pose and configuration cues; discarding them blinds the planner.
- **Reacher.** Low-dimensional continuous control; visual encoding of joint angle requires moderate resolution.
- **Cube.** Object pose and grasp-point spatial relations need moderate resolution, but Cube itself is largely insensitive to global noise (action sequence is structured; visual–action coupling is predictable).

Task-specificity is why input-side noise behaves as a coarse global pressure rather than a single jointly optimal level. Table 11 consolidates the behavioural (Section 5.3) and representational (Section 5.5) evidence into a single per-task read.

Table 11: Where each LeWM task sits under the selective-consistency tension. The two "demand" columns are qualitative task-property reads, not independent measurements; they index the underlying behavioural and representational evidence in Sections 5.3 and 5.5 and the operational definition in Section 4.3. Sweep signatures come from Tables 3 and 4; diagnostic readings from Table 6.

Task	Nuisance-contraction demand	Control-discriminability demand	Observed sweep signature	Diagnostic reading at representative checkpoint
TwoRoom	high (visual redundancy)	low–moderate (discrete navigation)	heavy-noise plateau ($\sigma^* = 0.08$)	compact latent acceptable; rank 47.6 → 37.7 while sweep success stays high
PushT	moderate	high (contact precision)	dissociated optima ($\sigma^*_{\mathrm{unp}} = 0.03$, $\sigma^*_{\mathrm{corr}} = 0.08$)	resolution must survive; noise training keeps rank/probe flat (76.4 → 77.4) with only a mild transition-resolution decline, whereas error-based reweighting collapses both (Section D)
Reacher	moderate	moderate	broad plateau ($\sigma^* \in 0.02$–0.07)	rollout drift carries weak residual signal more than rank or single-step resolution
Cube	low–moderate	moderate	weak response; pixels 0.08 point-best at $\sigma^*_{\mathrm{corr}} = 0.03$	structured action coupling buffers noise; small representation movement

Scope. The task-level fragility/recovery signature is replicated on PLDM, but the mechanism claim is LeWM-centred. Reconstruction-based world models, decoder-free latent MPC systems, EMA-target JEPAs, and variational JEPAs may exhibit different compression dynamics.

6.3 When the diagnostic toolkit works — and when it does not

Three conditions bound the toolkit. First, it can rank residual checkpoint quality after conditioning on `std_max`, but it does not isolate corruption-specific robustness: on PushT, fragility ratio retains signal for unperturbed and noisy success yet not for the unperturbed-to-corrupted gap. Second, tasks with little residual checkpoint spread after the sweep trend, such as Reacher and TwoRoom here, fall outside the strict diagnostic regime. Third, no static diagnostic can replace the training intervention itself; in this sweep, whether the model saw perturbations during training is the largest observed factor behind the corruption drop. The toolkit is therefore an unperturbed-evaluation auxiliary for mechanism localisation and checkpoint triage within a fixed protocol, not a robustness oracle.

6.4 Practical guidance: how to choose `std_max` on a new task

Use the diagnostics as a screening tool, then run direct eval at both unperturbed and high-corruption endpoints. Check fragility ratio, effective rank, transition resolution, and task semantics together; none is a robustness oracle alone. A practical search is a coarse sweep such as {0.01, 0.03, 0.05, 0.07}, followed by refinement around the best unperturbed/corrupted candidates.

6.5 A target-view scope boundary for future objectives

The completed prediction-target-view ablation is a negative scope check rather than a method contribution. It compares the retained full-sequence perturbed-target branch with perturbed-history → original-future prediction. The latter tests a tempting clean-target denoising variant, but Appendix H shows that this target change alone does not repair robustness: across the eight target-noise checkpoints, PushT remains at 6.75% at pixels 0.08 and Reacher remains at 19.62%, while the matched full-sequence branch reaches 72.83% and 76.17% respectively.

The result is consistent with a closed-loop train/eval consistency explanation rather than an eval-corruption implementation error. Training is a teacher-forced one-step denoising target (noisy real history $\rightarrow$ clean future), whereas CEM evaluation feeds each predicted latent back as the next autoregressive input. Whether the prediction target lives on the original or perturbed latent manifold therefore changes the next-step input distribution at evaluation time. This is not a causal proof that the target manifold is the only mechanism; a stronger claim would require a rollout-consistent denoising or multi-step objective. The next method step is therefore not another target-view toggle by itself, but adaptive predictive-dynamics consistency: regularise paired rollouts after the action-conditioned predictor and gate by action/transition sensitivity. The negative heteroscedastic result below is the warning: high prediction error can mark action-critical contact transitions, so prediction difficulty alone is the wrong gate.

6.6 Limitations and future directions

Limitation 1 — architecture scope. The main diagnostics are LeWM-centred; PLDM replicates task-level signatures, the PushT partial-correlation null, and the full Gaussian-noise ACPC basin pattern, but its internal recovery mechanism differs. EMA-target and variational JEPA variants remain open.

Limitation 2 — corruption scope. Gaussian pixel noise is the controlled training axis. Blur stress tests in Section F show visual fragility beyond Gaussian noise, but also corruption-specific task ordering; blur-specific training remains follow-up work.

Limitation 3 — diagnostic scope. The framework is empirical. Reacher and TwoRoom fail the partial-correlation criterion for all metrics, and we do not yet have a theory predicting which tasks support cross-checkpoint diagnostics.

Limitation 4 — method scope. The dense paired ACPC basin diagnostic now covers the LeWM and PLDM Gaussian-noise sweeps, but only for two related JEPA-family world models and one corruption family. We therefore use it as a replication of the paired-basin pattern, not as a method-invariant theorem. Appendix G broadens the probes under auxiliary pixels+goal stress, but those rows are post-hoc, small-sample, and exploratory. Appendix H adds the target-view ablation, which is negative; a method paper still needs a trained predictive-consistency objective.

Limitation 5 — training-run variability. The sweep reports variability over evaluation seeds, not independent training seeds. Each `std_max` cell corresponds to one trained epoch-10 checkpoint. The grid is therefore best interpreted as a controlled intervention sweep over training corruption strength, not as an estimate of training-run variability.

Additional ablation — hard $\neq$ nuisance. A heteroscedastic NLL probe with a learned per-transition σ-head shows why error-based downweighting is unsafe in this setting: TwoRoom remains strong, but PushT unperturbed success collapses from 86.33% to 13.33% because contact-control transitions are high-error and action-critical. Full data are in Section D.

Future direction 1 — adaptive predictive-dynamics training. Because the perturbed-history $\rightarrow$ unperturbed-future target ablation is negative by itself, full-sequence perturbed-target training remains the empirical baseline. The next objective is paired ACPC_H regularisation gated by action sensitivity with a discriminability guard.

Future direction 2 — broader replication. Frozen/pretrained visual-feature models, EMA-target JEPAs, and non-Gaussian training corruptions remain the next checks.

Future direction 3 — Theoretical grounding. Reformulating the consistency/discriminability tension in an information-bottleneck / rate-distortion language is a natural follow-up; we did not pursue it because the empirical phenomenology may not yet be stable enough for formal modelling.

7 Conclusion

This paper reframes visual robustness for JEPA world-model control as action-conditioned predictive consistency with an explicit discriminability countercondition. Controlled Gaussian corruptions show that latent prediction alone does not guarantee robustness; input-side noise augmentation recovers much of the lost control performance, but only as a coarse pressure with task-dependent plateaus. The paired ACPC basin diagnostic gives the control-facing reading: robust checkpoints induce tighter same-action prediction radii under same-state perturbations, while pointwise fragility does not isolate the corruption gap after conditioning on train-time noise.

We do not propose a new training algorithm. The target-view ablation in Appendix H rules out perturbed-history $\rightarrow$ original-future targets as a sufficient closed-loop fix and leaves full-sequence perturbed targets as the empirical mainline for this study. The remaining method path is to train paired predictive-dynamics consistency after the action-conditioned predictor, gated by action sensitivity and protected by discriminability.

Acknowledgements

The complete code and data are available at `https://github.com/qun-team/wm_exp`. We thank the LeWM authors for releasing the baseline framework on which this study builds.

References

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. ICLR 2017 Workshop Track / arXiv preprint, 2017.

[2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

[3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas

Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.

[5] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[6] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.

[7] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[8] Carlos E. García, David M. Prett, and Manfred Morari. Model predictive control: Theory and practice — a survey. *Automatica*, 25(3):335–348, 1989.

[9] Quentin Garrido, Randall Balestriero, Laurent Najman, and Yann LeCun. RankMe: Assessing the downstream performance of pretrained self-supervised representations by their rank. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10929–10974. PMLR, 2023.

[10] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

[11] Hafez Ghaemi, Eilif Benjamin Muller, and Shahab Bakhtiari. seq-jepa: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[12] Christopher Grimm, Andre Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

[13] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.

[14] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[15] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13611–13617, 2021.

[16] Yongchao Huang. Vjepa: Variational joint embedding predictive architectures as probabilistic world models. *arXiv preprint arXiv:2601.14354*, 2026.

[17] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[18] David Klindt, Yann LeCun, and Randall Balestriero. When does lejepa learn a world model?, 2026.

[19] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3519–3529. PMLR, 2019.

[20] Dirk P. Kroese, Sergey Porotsky, and Reuven Y. Rubinstein. The cross-entropy method for continuous multi-extremal optimization. *Methodology and Computing in Applied Probability*, 8(3):383–407, 2006.

[21] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[22] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026.

[23] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[24] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[25] Yuping Qiu, Rui Zhu, and Ying-cong Chen. Improving joint embedding predictive architecture with diffusion noise. *arXiv preprint arXiv:2507.15216*, 2025.

[26] Ashwath Radhachandran, Vedrana Ivezić, Shreeram Athreya, Ronit Anilkumar, Corey W. Arnold, and William Speier. Us-jepa: A joint embedding predictive architecture for medical ultrasound. *arXiv preprint arXiv:2602.19322*, 2026.

[27] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *Proceedings of the 15th European Signal Processing Conference (EUSIPCO)*, pages 606–610, 2007.

[28] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[29] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022.

[30] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *WRL@ICLR 2025*, 2025.

[31] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 20827–20840. PMLR, 2022.

[32] Alex Tamkin, Margalit Glasgow, Xiluo He, and Noah Goodman. Feature dropout: Revisiting the role of augmentations in contrastive learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[33] Jayden Teoh, Manan Tomar, Kwangjun Ahn, Edward S. Hu, Pratyusha Sharma, Akshay Krishnamurthy, Riashat Islam, Alex Lamb, and John Langford. Next-latent prediction transformers learn compact world models. *arXiv preprint arXiv:2511.05963*, 2025.

[34] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[35] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning, 2025.

[36] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.

[37] Tongzhou Wang and Phillip Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9929–9939. PMLR, 2020.

[38] Zijie Wang, Wei Zhang, Weiming Zhang, Fanqi Zhang, Xiao Tan, Yipeng Qin, and Guanbin Li. World models as group actions, 2026.

[39] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[40] Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[41] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[42] Junbo Zhang and Kaisheng Ma. Rethinking the augmentation module in contrastive learning: Learning hierarchical augmentation invariance with expanded views. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16650–16659, 2022.

A Experimental details

A.1 Environments

- **PushT.** 2D continuous pushing; 20,000 expert episodes; $\sim$196 steps/episode; action dim 2 (direction + magnitude); 224×224 RGB.
- **TwoRoom.** 2D continuous navigation; 10,000 episodes; $\sim$92 steps; action dim 2; 224×224 RGB.
- **Reacher.** 2D arm control (DeepMind Control Suite); 10,000 episodes; 200 steps; action dim 2.
- **Cube.** 3D cube manipulation (OGBench); 10,000 episodes; 200 steps; action dim 7.

A.2 Noise augmentation implementation

The actual implementation is `utils.py::AddNormalizedGaussianNoise`. Key points: (i) noise is added on ImageNet-normalized tensors and must be divided by the per-channel std to retain "pixel-space std" semantics; (ii) sampling is per-frame independent over the leading frame dims, not per-batch; (iii) all sweeps in this paper fix `noise_prob` = 1.0 and `std_min` = 0 and vary only `std_max`.

```
class AddNormalizedGaussianNoise:
    """Per-frame independent. Each frame draws Bernoulli(noise_prob) then
    std ~ Uniform(std_min, std_max). Pixel-space std is converted to
    normalized space by dividing by per-channel std before adding."""
    def __init__(self, std_min, std_max, noise_prob=1.0):
        self.std_low, self.std_high, self.noise_prob = std_min, std_max, noise_prob
        self.channel_std = torch.as_tensor(IMAGENET_STD) # (C,)

    def __call__(self, x):
        if self.std_high <= 0 or self.noise_prob <= 0:
            return x
        leading = x.shape[:-3]
        stds = torch.empty(leading, device=x.device, dtype=x.dtype).uniform_(
            self.std_low, self.std_high)
        if self.noise_prob < 1.0:
            mask = (torch.rand(leading, device=x.device) < self.noise_prob).to(x.dtype)
            stds = stds * mask
        per_frame_scale = stds.view(*leading, 1, 1, 1)
        channel_factor = (1.0 / self.channel_std.to(x.device, x.dtype)).view(
            *([1] * len(leading)), -1, 1, 1)
        scale = per_frame_scale * channel_factor # pixel-space std -> normalized
        return x + torch.randn_like(x) * scale
```

A.3 Evaluation protocol

Unperturbed evaluation uses no noise, 100 trajectories per seed × 3 seeds (42/43/44), totalling 300 trajectories per condition. The primary noisy evaluation adds Gaussian noise of std $\in \{0.05, 0.08\}$ to observation pixels while leaving the goal image unperturbed under the same 3-seed protocol. Pixels+goal corruption is retained as an auxiliary full-visual-stream stress condition and is reported separately where used. Success criterion is task-specific (PushT: T-block pose match; TwoRoom: target region; Reacher: joint-angle match; Cube: cube position match).

A.4 Compute

Training runs on a single NVIDIA H800 (80 GB) and follows the LeWM training schedule released with the baseline.

A.5 Main-figure rendering

Figures 2 and 4 are produced by `tools/paper1_figs.py`; run `python -m tools.paper1_figs -out-dir assets/paper1_figs`. The script reads `assets/paper1_data/canonical_evals_20260517.json` together with `assets/paper1_data/canonical_diagnostics_20260517.json`; no checkpoint or model loading is required.

Figure 3 is produced by `tools/paper1_selective_contraction.py`. The paper-facing rendering command is:

```
python -m tools.paper1_selective_contraction \
  --plot-clusters --plot-tasks PushT \
  --n-sequences 128 --cluster-anchor-count 16 \
  --view-stds 0.0 0.01 0.04 0.08 \
  --cluster-perturb-repeats 6 \
  --cluster-out-dir assets/paper1_figs \
  --cluster-envelope ellipse --cluster-envelope-coverage 0.90
```

Unlike the aggregate figure generator, this optional rendering path loads the PushT checkpoints and dataset windows to draw repeated same-state perturbation samples. The panel annotations are high-dimensional statistics; the t-SNE ellipses are only qualitative 2-D covariance envelopes.

A.6 Local cluster atlas: a projection-free check of the basin visualization

Because t-SNE distorts global distances, Figure 5 provides a complementary *local normalized atlas* view of the same PushT data. Each small box selects one original dataset state; the circle radius equals the distance to its nearest *other* original state, computed in the original high-dimensional feature space, and the coloured markers are that state's original and Gaussian-perturbed views after a local PCA used only for in-box display. The visual question per box is therefore exactly the high-dimensional one: does the same-state perturbation cloud stay inside the nearest-different-state radius? On the clean-trained checkpoint many perturbation views escape their circles (median $r/\mathrm{NN} > 1$, few disjoint balls); on the full-sequence noise-trained checkpoint the clouds contract well inside the circles in both encoder and predictor-rollout features. This avoids the global-projection caveats of Figure 3 at the cost of showing only local neighbourhoods; the quantitative multi-task evidence remains Table 5.

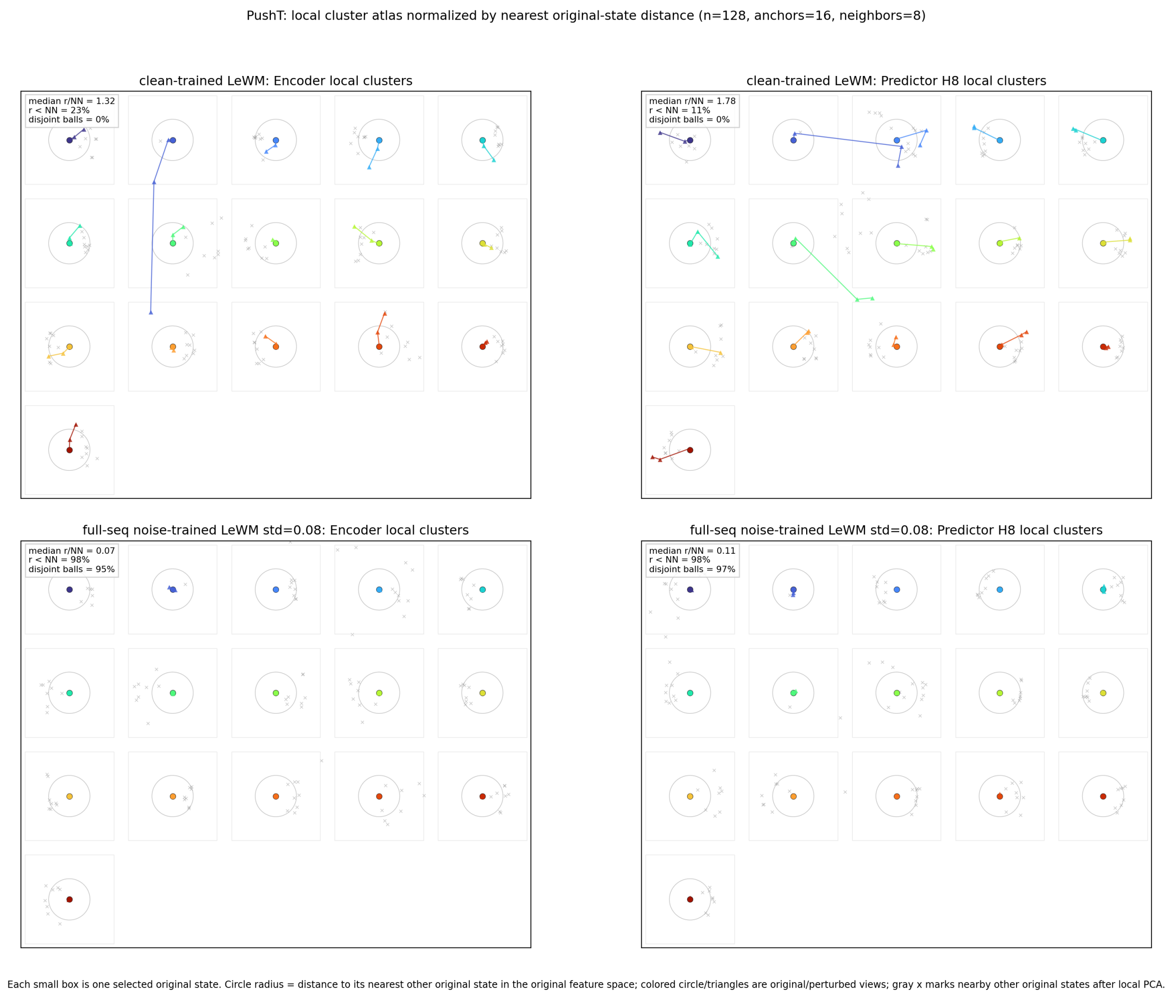


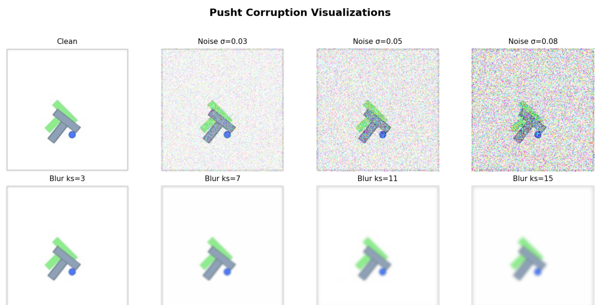
Figure 5: Local cluster atlas for PushT, normalized by nearest original-state distance ($n = 128$ windows, 16 anchors, 8 neighbours). Top: clean-trained LeWM; bottom: full-sequence noise-trained LeWM at `std_max` $= 0.08$. Left: encoder-history features; right: $H = 8$ predictor-rollout features. Each box is one selected original state; the circle radius is the high-dimensional distance to its nearest other original state; coloured circles/triangles are the original/perturbed views; grey crosses are nearby other original states after local PCA. If triangles stay inside the circle, same-state perturbations remain closer than the nearest different state under that local high-dimensional normalization. Inset statistics (*median* r/NN, $r <$ NN, *disjoint balls*) are computed in the original high-dimensional feature space.

The atlas rendering command is:

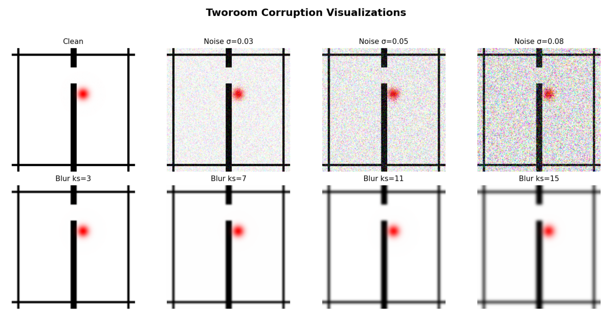
```
python -m tools.paper1_selective_contraction \
  --plot-atlas --plot-tasks PushT \
  --n-sequences 128 --atlas-anchor-count 16 \
  --view-stds 0.0 0.01 0.04 0.08 \
  --atlas-out-dir assets/paper1_figs
```

A.7 Corruption visualizations

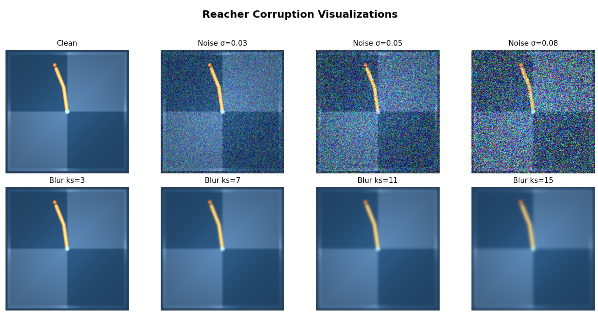
Figure 6 shows representative renderings of the two evaluated corruption families, applied to a sample observation from each of the four tasks. The Gaussian-noise row includes a mild calibration severity $\sigma = 0.03$ in addition to the two evaluation endpoints $\sigma \in \{0.05, 0.08\}$ used in Sections 5.2 and 5.3; the main sweep treats observation-only noise with an unperturbed goal as the primary endpoint and keeps pixels+goal noise as an auxiliary stress condition. The Gaussian-blur kernel sizes ($k \in \{3, 7, 11, 15\}$) match the stress test in Section F. Each panel is a single 224×224 RGB frame from the corresponding task's evaluation rollout, after corruption and before encoder normalisation.



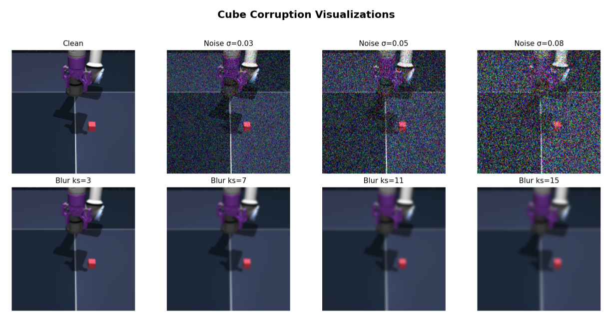
(a) PushT



(b) TwoRoom



(c) Reacher



(d) Cube

Figure 6: Corruption visualizations on the four tasks. Each panel is a 2×4 grid: additive Gaussian pixel noise on row 1 (Original, $\sigma \in \{0.03, 0.05, 0.08\}$) and Gaussian blur on row 2 ($k \in \{3, 7, 11, 15\}$, auto sigma). The $\sigma = 0.08$ endpoint is the primary Gaussian stress condition used in Tables 2 and 4.

B Full diagnostic profile of LeWM-base on four tasks

Table 12 summarises every core diagnostic on the four LeWM-base checkpoints. Data are taken from each checkpoint's per-tool JSON outputs under `eval_results/diagnostics/` (the `geometry`, `task`, `predictor` and `latent_noise` families).

Table 12: Full LeWM-base diagnostics across four tasks.

Layer / Metric	TwoRoom	PushT	Reacher	Cube	Unit / note
Encoder geometry					
`clean_nn_cos_dist_median`	0.0449	0.2360	0.0633	0.1856	cosine distance
`clean_pair_cos_dist_median`	0.9904	1.0228	1.0252	1.0193	pair-wise cos dist
`clean_effective_rank`	47.60	76.42	61.04	73.25	effective rank
Noise sensitivity					
`noise_angle_deg_median` (@std= 0.05)	5.51°	1.33°	3.22°	1.40°	median angular shift
`noise_to_nn_cos_ratio_median`	0.1031	0.011	0.0249	0.016	noise/NN cos ratio
`robust_radius_std`	0.0142	0.0537	0.0142	0.0356	critical noise std
`first_risk_std`	>0.08	>0.08	>0.08	>0.08	first high-risk std
`noise_angle_slope_deg_per_std`	1085.8	284.8	831.7	327.0	°/std angular gain
`geometry_flag`	balanced	robust	balanced	robust	geometry label
Task resolution					
`transition_resolution_ratio_l2`	0.7216	0.3015	0.3704	0.4847	L2 resolution ratio
`transition_resolution_ratio_cos`	0.5538	0.0868	0.1351	0.2347	cos resolution ratio
`id_probe_r2`	0.2889	0.7739	0.1621	0.6657	linear probe R^2
`id_probe_r2_min`	0.2599	0.6786	0.1366	0.0972	min probe R^2
`lidar_rank`	46.06	13.95	45.90	42.46	LiDAR rank proxy
Action effect					
`action_mean_pred_shift_norm`	0.5329	0.1283	0.2518	0.2364	action-perturb mean shift
`action_perturb_pred_shift_corr`	0.2847	0.2873	0.4042	0.2559	shift $\times$ $\lVert a \rVert$ corr
Predictor stability					
`predictor_rollout_T8_l2`	18.62	18.65	15.17	20.20	T=8 rollout L2 drift
`predictor_target_to_nn_cos_ratio_at_max_std`	1.51e-4	3.54e-6	2.67e-5	3.39e-6	max std target/NN ratio
Latent noise					
`cka_linear_at_max_std`	0.1986	0.5536	0.3085	0.1814	CKA unperturbed vs noisy
`latent_cost_surface_slope_z`	635.31	1.3886	599.45	0.6208	goal latent cost slope

C Heteroscedastic-loss formulation

The scale-preserving heteroscedastic NLL referenced in Section 6.6 (the "Additional ablation" paragraph) and detailed in Section D is:

$$\mathcal{L}_{\text{hetero}} = \tfrac{1}{2}\,\exp(-s_t)\,\lVert z_{t+1} - \hat{z}_{t+1} \rVert^2 + \tfrac{1}{2}\,s_t\,, \tag{8}$$

where s_t is the σ-head-predicted log-variance. During training $\exp(-s_t)$ acts as an automatic weight: high-error transitions are down-weighted and low-error transitions are up-weighted. When $s_t \equiv 0$ the loss reduces to plain MSE.

D Heteroscedastic-loss negative result (data)

This appendix records the full data behind the "Additional ablation" paragraph in Section 6.6. The σ-head is trained jointly with the predictor; gradient is detached on the σ path so the mean prediction path is exactly LeWM MSE when σ is constant.

Table 13: Heteroscedastic-loss evaluation (canonical 3-seed $\times$ 100 protocol). The LeWM+noise rows are each task's pixels+goal 0.08 point-best checkpoint (TwoRoom `std_max` = 0.08, PushT `std_max` = 0.06), matching the strongest stress column of this ablation; the observation-only point-bests in Table 4 differ.

Task / model	Unperturbed	goal 0.05	pixels 0.05	px+goal 0.05	goal 0.08	px+goal 0.08
TwoRoom LeWM-base	94.00	73.33	72.33	61.33	58.67	50.00
TwoRoom LeWM+noise point-best	98.33	98.00	98.33	98.00	98.67	98.67
TwoRoom hetero	99.67	85.33	96.67	84.67	73.33	55.33
PushT LeWM-base	86.33	38.00	17.00	12.00	11.33	4.67
PushT LeWM+noise point-best	89.33	87.67	88.00	88.33	89.67	87.00
PushT hetero	13.33	7.67	7.67	7.67	9.67	6.00

TwoRoom hetero reaches 99.67% on unperturbed evaluation — consistent with the prior that low-dimensional discrete tasks benefit from stronger nuisance contraction / clustering — but lags noise training on high-noise robustness. **PushT hetero unperturbed success is** 13.33% **— a method-level failure**, not evidence for a useful robustness compromise.

Table 14: Representation diagnostics of the hetero-loss failure.

Metric	TwoRoom base	TwoRoom hetero	PushT base	PushT hetero
`clean_nn_cos_dist_median`	0.0449	0.0281	0.2360	0.1051
`clean_effective_rank`	47.60	33.59	76.42	42.85
`transition_resolution_ratio__cos`	0.5538	0.3780	0.0868	0.0101
`transition_resolution_ratio__l2`	0.7216	0.6055	0.3015	0.1023
`id_probe_r2`	0.2889	−0.0573	0.7739	0.2678
`action_mean_pred_shift_norm`	0.5329	0.4482	0.1283	0.0841
`predictor_rollout_T8_l2`	18.62	17.90	18.65	14.01

Hetero-loss compresses the representation in both tasks. For TwoRoom (low-dimensional, discrete, redundant), compression is acceptable. For PushT, the L2 transition-resolution ratio collapses from 0.3015 to 0.1023 and `id_probe_r`^{2} from 0.7739 to 0.2678 — task-relevant state information is erased. The contrast with input-side noise training is sharp: the PushT noise-sweep representative in Table 6 leaves rank and probe nearly unchanged, so this collapse is specific to error-based downweighting. The drop in multi-step rollout drift is not good news either: the latent has become more *predictable* without being more *controllable*.

Mechanism. The σ-head correctly learns per-transition difficulty (high σ on contact frames in PushT, on doorway crossings in TwoRoom, etc.), but using σ as an automatic weight to down-weight high-error transitions misclassifies the contact-and-control-critical states of PushT as "unimportant" and erases them. This is a negative example of the broader selective-consistency lesson: in contact-heavy control, **hard $\neq$ unimportant**.

This finding motivates a follow-up direction we are currently exploring: per-token *consistency* (not loss-reweighting) controllers driven by detached difficulty signals, which preserve the mean prediction path's gradient distribution. That work is independent of this paper's diagnostic study and will be reported separately.

E Cross-method replication: PLDM noise sweep on four tasks

This appendix records the second method-family on which the noise sweep and the cross-checkpoint correlation analysis replicate. PLDM follows the joint-embedding predictive line from Sobal et al. [29]; the concrete latent-dynamics planning baseline is from Sobal et al. [30]. We use the implementation distributed through the `stable-worldmodel` baseline suite [22]. Training and evaluation follow the LeWM canonical protocol bit-for-bit: per-frame Gaussian pixel noise via `utils.py::AddNormalizedGaussianNoise`, `std_max` $\in \{0.01, \ldots, 0.08\}$, three evaluation seeds (42/43/44), 100 trajectories per seed.

Coverage. The PLDM release is complete for the same grid as the LeWM canonical sweep: 4 tasks $\times$ 9 configurations (no-noise baseline plus `std_max` $\in \{0.01, \ldots, 0.08\}$), three evaluation seeds (42/43/44), and 100 trajectories per seed. The PLDM eval, predictor-diagnostic, full-diagnostic, and cross-method-correlation JSON files are listed in the data manifest; we do *not* merge them into the LeWM canonical files used by Tables 2–9.

No-noise-trained PLDM cliff. PLDM reproduces the no-noise-trained visual-corruption cliff on PushT and TwoRoom, while Reacher and Cube show much smaller drops under this PLDM training recipe (Table 15). This supports a method-family reading of the failure mode without claiming every task has the same cliff magnitude.

Table 15: PLDM baseline trained without input-noise augmentation on all four tasks. Success rate mean $\pm$ population std, 3 seeds $\times$ 100 trajectories.

Task	unperturbed	pixels 0.05	pixels 0.08	px+goal 0.08	unperturbed $\to$ pixels 0.08 drop
TwoRoom	96.67 ± 1.70	79.67 ± 4.03	67.00 ± 2.16	51.67 ± 2.05	-29.67
PushT	75.33 ± 3.68	46.00 ± 4.97	18.33 ± 2.05	10.00 ± 2.16	-57.00
Reacher	82.67 ± 1.70	81.33 ± 4.03	80.33 ± 5.73	79.33 ± 0.94	-2.33
Cube	52.67 ± 3.30	50.33 ± 4.50	48.33 ± 4.50	48.33 ± 2.87	-4.33

PLDM full sweep — unperturbed evaluation. Table 16 reports the per-task unperturbed success rate at every trained `std_max` level. Table 17 reports the corresponding pixels 0.08 robustness with an unperturbed goal.

Table 16: PLDM+noise sweep, unperturbed success rate (%; released condition-name: `clean`). † marks the per-task unperturbed point-best.

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	96.67 ± 1.70	75.33 ± 3.68	82.67 ± 1.70	52.67 ± 3.30
0.01	96.33 ± 0.94	72.33 ± 4.19	78.00 ± 0.82	51.33 ± 1.25
0.02	97.33 ± 0.47	71.33 ± 3.86	82.33 ± 2.36	53.33 ± 1.70
0.03	96.67 ± 1.70	$76.67 \pm 7.54^{\dagger}$	83.00 ± 3.74	52.67 ± 3.30
0.04	97.67 ± 0.47	68.67 ± 5.25	$85.00 \pm 2.16^{\dagger}$	54.00 ± 2.94
0.05	97.67 ± 1.25	74.33 ± 3.40	72.00 ± 1.41	54.00 ± 2.16
0.06	98.00 ± 0.82	70.33 ± 6.18	79.00 ± 0.82	$56.00 \pm 4.97^{\dagger}$
0.07	98.00 ± 0.82	66.67 ± 8.38	80.67 ± 2.62	53.67 ± 3.68
0.08	$98.33 \pm 0.94^{\dagger}$	68.00 ± 1.41	80.33 ± 1.25	54.00 ± 1.63

Table 17: PLDM+noise sweep, pixels 0.08 success rate with unperturbed goal (%). ‡ marks the per-task pixels 0.08 point-best.

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	67.00 ± 2.16	18.33 ± 2.05	80.33 ± 5.73	48.33 ± 4.50
0.01	96.00 ± 1.63	23.33 ± 2.49	77.67 ± 3.40	51.33 ± 2.36
0.02	95.67 ± 0.94	47.00 ± 0.82	80.67 ± 5.25	51.00 ± 4.32
0.03	96.33 ± 1.25	$73.00 \pm 7.48^{\ddagger}$	$81.67 \pm 4.71^{\ddagger}$	54.33 ± 2.62
0.04	97.33 ± 0.94	66.67 ± 5.73	80.33 ± 2.87	$54.67 \pm 2.36^{\ddagger}$
0.05	97.67 ± 1.89	71.67 ± 5.79	62.67 ± 4.19	54.00 ± 3.74
0.06	$98.00 \pm 0.00^{\ddagger}$	69.33 ± 4.78	75.00 ± 0.82	54.33 ± 0.94
0.07	98.00 ± 0.82	64.00 ± 7.79	78.33 ± 2.05	53.00 ± 3.74
0.08	97.67 ± 1.25	68.00 ± 5.66	79.33 ± 1.25	53.67 ± 2.05

Reading. Four quantitative observations are useful for interpreting PLDM as a second method family:

1. **TwoRoom (visually redundant) benefits from heavy noise on both methods.** PLDM pixels 0.08 success rises from 67.00% at the no-noise baseline to 98.00% at `std_max` $= 0.06$, then remains on a high-noise plateau. This mirrors the LeWM reading that TwoRoom tolerates strong same-state nuisance contraction.
2. **PushT (contact-heavy) exhibits the corruption cliff and large noise-training recovery.** PLDM without noise augmentation drops by 57.00 pt under pixels 0.08; training with `std_max` $= 0.03$ recovers pixels 0.08 success to 73.00% (+54.67 pt over the no-noise baseline). The unperturbed and pixels 0.08 point-bests are co-located at `std_max` $= 0.03$ on PLDM.
3. **Reacher is already robust under no-noise-trained PLDM.** The no-noise PLDM drop is only 2.33 pt, and the observation-noise point-best sits at `std_max` $= 0.03$ (81.67%). We therefore treat Reacher as a low-cliff external baseline rather than evidence for a universal noise-training gain.
4. **Cube (structured 3D manipulation) remains weakly noise-responsive.** PLDM Cube unperturbed success spans 51.33–56.00% and pixels 0.08 spans 48.33–54.67% across the full grid. This is the same weak noise responsiveness seen on LeWM.

Method-specific details. PLDM's PushT unperturbed point-best (76.67% at `std_max` $= 0.03$) is lower than LeWM's (89.67% at `std_max` $= 0.03$), so the external baseline is not a performance leaderboard. It is a robustness check: the cliff-and-recovery shape is reproduced, while the exact unperturbed/robust optimum location is method-dependent. In particular, the LeWM PushT optimum dissociation should be read as a LeWM-family signature, not a cross-method law.

Cross-method partial correlations. We repeat the fragility ratio partial-correlation analysis for PLDM on all four tasks. The headline PushT null replicates in the limited sense that we do not see stable evidence for a non-zero residual corruption-gap association: PLDM has unconditional $\rho(\text{metric}, \text{corruption drop}) = -0.75$ but partial $\rho(\text{metric}, \text{corruption drop}) \mid$ `std_max` $= -0.05$ (95% bootstrap CI $[-0.92, +0.61]$), and the joint LeWM+PLDM PushT partial after conditioning on `std_max` and method is $+0.22$ (95% CI $[-0.59, +0.61]$). The intervals are wide and include zero. Other tasks show task-specific residuals, so we use the full table as a scope boundary rather than as a universal diagnostic claim.

Table 18: PLDM fragility ratio partial correlations, $n = 9$ per task. The first three partial columns condition on `std_max` within PLDM; the final column uses the joint LeWM+PLDM sample ($n = 18$) and conditions on both `std_max` and a method dummy. Numbers come from the released PLDM correlation JSON.

Task	PLDM n	unperturbed partial	pixels 0.08 partial	corruption-drop partial	joint corruption-drop partial
TwoRoom	9	−0.26	−0.43	+0.35	+0.12
PushT	9	−0.22	−0.13	−0.05	+0.22
Reacher	9	−0.68	−0.49	+0.17	+0.43
Cube	9	−0.36	+0.19	−0.53	−0.13

PLDM full-sweep ACPC basin replication. We also run the Gaussian-noise basin protocol on all 36 PLDM checkpoints. Table 19 reports the compact baseline-vs-pixels-0.08-point-best summary, but the released source contains the full 4 tasks × 9 configs grid: `acpc_basin_diagnostics_pldm.json`.

Table 19: PLDM full-sweep Gaussian-noise ACPC basin replication, summarised by the no-noise baseline and each task's PLDM pixels 0.08 point-best checkpoint. The protocol matches Table 5: same-state Gaussian-noise views of the observation history (std 0.01–0.08), clean goal, and the same recorded action sequence.

Task	checkpoint	pixels 0.08	drop	R_E	R_F
TwoRoom	base	67.00	29.67	3.287	0.611
TwoRoom	noise 0.06	98.00	0.00	0.235	0.034
PushT	base	18.33	57.00	0.874	0.846
PushT	noise 0.03	73.00	3.67	0.153	0.091
Reacher	base	80.33	2.33	0.237	0.060
Reacher	noise 0.03	81.67	1.33	0.135	0.039
Cube	base	48.33	4.33	0.831	0.453
Cube	noise 0.04	54.67	−0.67	0.146	0.054

Reading. The PLDM basin replication is directionally consistent with the LeWM ACPC-basin reading: the pixels 0.08 point-best checkpoint reduces R_F on all four tasks, sharply on TwoRoom, PushT, and Cube. Reacher is the scope boundary: PLDM is already near-robust at the no-noise baseline, and both the behavioural drop and the basin change are small. The full 36-row artifact reduces the concern that the paired basin pattern is purely a LeWM artifact, but it does not upgrade the claim to a method-invariant mechanism because Table 20 shows different internal diagnostic movement.

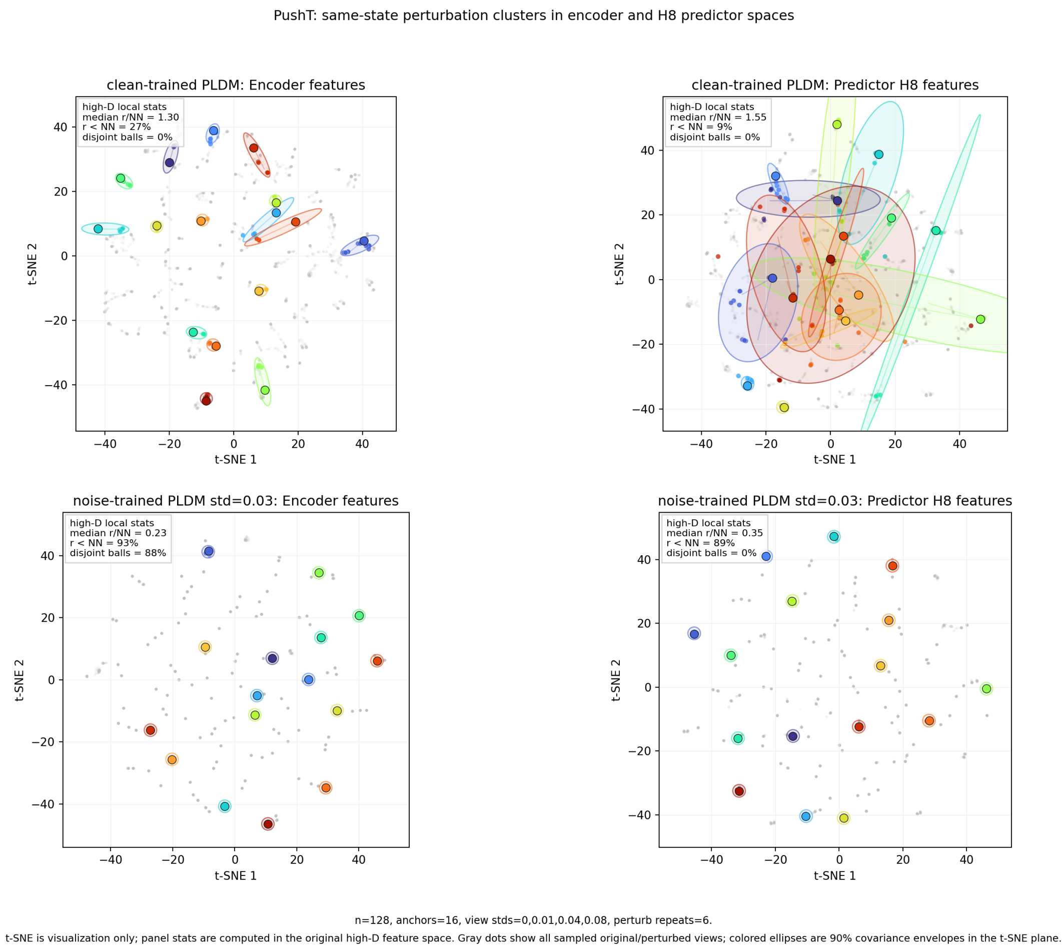


Figure 7: Qualitative PLDM PushT view of same-state perturbation samples for Table 19, rendered with the protocol of Figure 3 (top: clean-trained PLDM; bottom: noise-trained PLDM at `std_max` $= 0.03$). This is a qualitative method-family check only: t-SNE coordinates and envelope areas are not comparable with LeWM, and the quantitative evidence is the high-dimensional ACPC basin artifact summarised in Table 19.

PLDM full-diagnostic mechanism check. We also run the same five diagnostic layers on the PLDM sweep and release the resulting per-checkpoint summary in `canonical_full_diagnostics_pldm_20260523.json`. The compact base-vs-representative view in Table 20 matches the dominant LeWM movement in Table 6: representative recovery checkpoints on both families preserve rank, transition resolution, and the inverse-dynamics probe to first order, while strongly reducing multi-step predictor drift. The clearest method-specific difference is TwoRoom: LeWM's heavy-noise representative also compresses effective rank ($47.6 \rightarrow 37.7$), whereas PLDM's representative leaves rank essentially unchanged ($74.8 \rightarrow 72.7$). This separates two claims: the task-level fragility/recovery signature replicates across method families, while the residual diagnostic movement remains architecture-dependent.

Table 20: PLDM full-diagnostic check, no-noise baseline → representative noise-trained checkpoint. Representatives are each task's PLDM pixels 0.08 point-best checkpoint (TwoRoom `std_max` = 0.06, PushT `std_max` = 0.03, Reacher `std_max` = 0.03, Cube `std_max` = 0.04). Values come from the released full-diagnostics JSON.

Task	representative `std_max`	effective rank	transition ratio L2	inverse-dynamics probe R^2	rollout T8 L2
TwoRoom	0.06	74.78 → 72.70	0.8188 → 0.8254	0.3233 → 0.3201	20.36 → 1.02
PushT	0.03	130.13 → 131.90	0.4710 → 0.4778	0.7570 → 0.7479	20.25 → 2.82
Reacher	0.03	117.52 → 108.14	0.5053 → 0.4824	0.2150 → 0.1844	1.45 → 0.94
Cube	0.04	134.80 → 124.49	0.6395 → 0.6313	0.5428 → 0.5576	18.98 → 4.02

What this strengthens, and what it does not. PLDM strengthens the empirical evidence by showing that the four task signatures are not tied to one LeWM checkpoint family. The full PLDM basin artifact, summarised in Table 19, further supports the paired same-state perturbation reading at the baseline-vs-robust endpoints. It also strengthens the diagnostic-boundary claim: after removing the sweep-level `std_max` effect and the method offset on PushT, the local predictor-sensitivity diagnostic does not isolate corruption-specific robustness. The TwoRoom and Cube PLDM residuals are deliberately left visible in Table 18. They do not contradict the headline null: the intervals are wide, the noisy-eval ranges are small on the saturated tasks, and the residuals should not be converted into a robustness oracle.

Mechanism boundary. Table 20 is the reason we do not promote any single internal mechanism to a universal theorem. On both families the most consistent representative-checkpoint movement is a strong reduction in multi-step rollout drift, but the residual movement is method-specific: LeWM compresses TwoRoom's effective rank where PLDM does not, and the two families place their weak residual partial-correlation signals on different tasks (Tables 8 and 18). The shared conclusion is therefore about *control-time visual fragility and noise-training recovery*; the internal diagnostic route is method-specific.

F Cross-corruption sanity check: no-noise-trained blur evaluation

This appendix addresses the narrow concern that the observed visual fragility is an artefact of Gaussian noise only. We evaluate the LeWM and PLDM baselines trained without input-noise augmentation under Gaussian blur with kernel sizes $3, 7, 11, 15$, applied to pixels, goal images, or both. This is an **eval-only** stress test: no blur-trained checkpoint is included, and the blur data are not mixed into the Gaussian-noise sweep. Source: `canonical_blur_baselines_20260523.json`.

Table 21: Pixels+goal blur stress test on baselines trained without input-noise augmentation. Success rate mean ± population std over 3 evaluation seeds × 100 trajectories. Kernel sizes 11 and 15 are severe low-pass endpoints on 224×224 inputs, so they should not be read as mild corruptions.

Method	Task	unperturbed	$k = 3$	$k = 7$	$k = 11$	$k = 15$	worst drop
LeWM	TwoRoom	94.00 ± 3.56	39.33 ± 2.05	32.67 ± 3.09	30.33 ± 2.87	30.67 ± 0.47	-63.67
LeWM	PushT	86.33 ± 2.36	81.67 ± 4.50	74.00 ± 1.41	65.33 ± 2.49	58.67 ± 6.65	-27.67
LeWM	Reacher	58.67 ± 1.25	57.00 ± 6.38	45.67 ± 2.05	36.00 ± 4.90	20.33 ± 2.49	-38.33
LeWM	Cube	66.67 ± 2.62	65.00 ± 1.63	62.33 ± 2.62	57.33 ± 2.87	54.00 ± 2.16	-12.67
PLDM	TwoRoom	96.67 ± 1.70	38.33 ± 0.47	30.33 ± 2.87	28.33 ± 1.25	28.33 ± 1.70	-68.33
PLDM	PushT	75.33 ± 3.68	74.00 ± 2.94	72.00 ± 3.56	67.67 ± 4.03	62.33 ± 2.05	-13.00
PLDM	Reacher	82.67 ± 1.70	86.00 ± 2.16	80.00 ± 3.56	72.00 ± 2.16	68.00 ± 4.08	-14.67
PLDM	Cube	52.67 ± 3.30	53.00 ± 5.35	53.00 ± 2.83	50.67 ± 3.68	52.33 ± 5.44	-2.00

Reading. Blur is a different failure mode from Gaussian pixel noise. The clear collapse is TwoRoom: it falls below 40% already at $k = 3$ on both LeWM and PLDM and then saturates around 28–33% for $k = 7$–15. This reverses the Gaussian-noise ordering, where PushT is the most fragile task. The other tasks are more stable under blur: PLDM PushT, PLDM Reacher, and both Cube baselines lose at most about 15 pt at the strongest endpoint, while LeWM PushT degrades gradually and LeWM Reacher drops sharply only at $k = 15$. We therefore use blur only as an eval-only cross-corruption sanity check: visual fragility is not unique to Gaussian noise, but the task ordering and recovery profile are corruption-specific.

G Phase-0 paired ACPC diagnostics

This appendix records the first full-coverage run of the paired action-conditioned diagnostics defined in Section 3.2. The artifact `assets/paper1_data/acpc_phase0_diagnostics.json` covers 72 checkpoint rows: LeWM and PLDM, four tasks, and nine training-noise levels per task. All rows completed successfully. Each row uses 100 sampled sequences, Gaussian pixels+goal corruption at std 0.08, rollout horizon $H = 8$, and a shared candidate-action set of one dataset future plus 64 random futures. Because this uses the auxiliary pixels+goal stress endpoint rather than the paper's primary unperturbed-goal endpoint, it is reported as a broader exploratory sanity check rather than as the main ACPC evidence. PCC, CRA, and MAF are computed from the same original/noisy candidate-cost vectors. ADM and SPRR use an action-distance latent proxy, not an oracle state/keypoint margin.

Table 22: Phase-0 paired ACPC diagnostics, no-noise baseline → px+goal 0.08 point-best checkpoint. Lower is better for ACPC-H/transition, PCC, and MAF; higher is better for CRA and top-8 overlap. Values are exploratory diagnostics, not predictor-selection rules.

Task	Method	robust `std_max`	px+goal 0.08	ACPC-H/transition	PCC median	CRA mean	top-8 overlap	MAF flip
TwoRoom	LeWM	0.08	$50.00 \to 98.67$	$1.254 \to 0.042$	$176.6 \to 7.0$	$0.157 \to 0.990$	$0.201 \to 0.952$	$0.92 \to 0.06$
TwoRoom	PLDM	0.05	$51.67 \to 98.33$	$1.062 \to 0.063$	$164.3 \to 7.1$	$0.367 \to 0.982$	$0.287 \to 0.911$	$0.85 \to 0.05$
PushT	LeWM	0.06	$4.67 \to 87.00$	$2.740 \to 0.149$	$67.3 \to 4.6$	$0.323 \to 0.986$	$0.213 \to 0.914$	$0.96 \to 0.08$
PushT	PLDM	0.03	$10.00 \to 72.00$	$1.682 \to 0.162$	$52.0 \to 8.1$	$0.300 \to 0.967$	$0.286 \to 0.869$	$0.90 \to 0.08$
Reacher	LeWM	0.02	$15.00 \to 85.67$	$1.922 \to 0.023$	$528.0 \to 1.2$	$0.242 \to 0.999$	$0.219 \to 0.990$	$0.97 \to 0.01$
Reacher	PLDM	0.04	$79.33 \to 81.33$	$0.104 \to 0.063$	$11.7 \to 6.3$	$0.965 \to 0.987$	$0.930 \to 0.949$	$0.13 \to 0.10$
Cube	LeWM	0.07	$46.33 \to 68.00$	$1.944 \to 0.055$	$88.8 \to 4.1$	$0.337 \to 0.996$	$0.266 \to 0.951$	$0.88 \to 0.00$
Cube	PLDM	0.08	$48.33 \to 56.33$	$1.136 \to 0.034$	$113.6 \to 3.0$	$0.663 \to 0.997$	$0.512 \to 0.967$	$0.74 \to 0.01$

Reading. The Phase-0 pass converts the ACPC definitions from a prospective protocol into a release artifact that behaves sensibly on the existing sweep. The largest Gaussian-noise cliffs (TwoRoom and PushT on both methods, Reacher/Cube on LeWM) coincide with large baseline ACPC-H, PCC, ranking disruption, and action flips, and the px+goal point-best checkpoints reduce these quantities sharply. PLDM Reacher is the important scope boundary: it already has high px+goal success at the no-noise baseline, and the paired diagnostics are correspondingly already close to the recovered checkpoint. Across the joint LeWM+PLDM sample within each task, partial Spearman correlations after conditioning on `std_max` and method remain task-specific: ACPC-H/transition vs. corruption drop is strong on PushT (+0.67) and Reacher (+0.71), but only modest on TwoRoom (+0.21) and Cube (+0.33). We therefore treat Table 22 as a face-validity and mechanism-localisation check, not as evidence that ACPC-H, PCC, CRA, MAF, ADM, or SPRR can by themselves predict robustness.

H Target-view ablation completion check

This appendix records the completed LeWM target-view ablation motivated in Section 6.5. The runs use `loss.pred.target_view=origin`: the prediction context is encoded from the configured Gaussian-perturbed history, while the prediction target remains the original future latent. This is the perturbed-history → original-future branch, contrasted with the default full-sequence perturbed-target branch used by the main LeWM+noise sweep. The artifact `assets/paper1_data/target_view_closed_loop_summary.json` records the aggregate values.

We verified the four-task run set after completion: eight target-noise checkpoints per task and epoch-10 evaluation summaries for `origin`, `pixels_std0.03`, `pixels_std0.05`, and `pixels_std0.08`. The canonical table below uses seeds 42/43/44 and 100 trajectories per seed.

Table 23: LeWM perturbed-history → original-future target-view ablation. Values are success rate mean ± population std over the eight target-noise checkpoints; each checkpoint value is the three-seed mean from `eval_summary.csv`.

Task	unperturbed	pixels 0.03	pixels 0.05	pixels 0.08
TwoRoom	93.33 ± 1.72	88.67 ± 6.04	77.71 ± 9.12	61.75 ± 7.65
PushT	82.92 ± 4.00	57.71 ± 8.96	20.21 ± 11.68	6.75 ± 5.00
Reacher	59.92 ± 1.80	40.38 ± 2.78	32.00 ± 4.57	19.62 ± 3.97
Cube	65.00 ± 2.01	61.42 ± 2.06	44.29 ± 1.32	39.62 ± 1.22

Table 24: Closed-loop target-view comparison at eval pixels 0.08. Values are success rate mean ± population std over the eight training-noise checkpoints; each checkpoint value is the canonical three-seed mean.

Task	full-sequence perturbed target	perturbed-history → original future	full-seq advantage
TwoRoom	94.71 ± 1.90	61.75 ± 7.65	+32.96
PushT	72.83 ± 12.84	6.75 ± 5.00	+66.08
Reacher	76.17 ± 10.89	19.62 ± 3.97	+56.54
Cube	59.83 ± 5.55	39.62 ± 1.22	+20.21

Reading. The ablation acts as a scope boundary and supports retaining the full-sequence empirical mainline. Simply changing the prediction target to the original future does not reproduce the recovery from input-side noise augmentation in Table 4: PushT and Reacher remain close to their no-noise robustness cliffs, TwoRoom degrades under stronger eval noise despite high unperturbed success, and Cube does not improve over the main sweep.

A matched PushT 0to008 check shows the contrast: rerunning both branches' `std_max` $= 0.08$ checkpoints under the same fixed evaluation code and canonical seeds gives pixels 0.08 three-seed means of 86.33% for the full-sequence checkpoint versus 8.67% for the origin-target checkpoint (the canonical sweep value for the same full-sequence checkpoint in Table 4 is 89.00%; the difference is normal evaluation variance). We interpret this as evidence consistent with a closed-loop train/eval consistency explanation: full-sequence noise augmentation better matches the planner's latent autoregressive feedback, while one-step origin-target denoising does not. This is not evidence that arbitrary history mismatches are harmless; it says the target view alone is not the missing mechanism, and an explicit action-conditioned predictive-consistency objective is still needed.